1. INTRODUCTION

1.1 ABOUT THE PROJECT

In modern healthcare management, providing the general public with accurate, timely, and accessible medical guidance remains a critical challenge. When experiencing early physical discomfort or ambiguous health symptoms, individuals overwhelmingly resort to public search engines. This behavior frequently precipitates a documented behavioral phenomenon known as cyberchondria, wherein users experience heightened medical anxiety driven by search algorithms that inherently prioritize catastrophic or rare diagnoses over common, benign conditions. Conversely, patients presenting with atypical or subtle manifestations of acute, life-threatening conditions, such as silent myocardial infarctions, transient ischemic attacks, or early pulmonary embolisms, often fail to recognize the gravity of their symptoms, leading to dangerous delays in seeking emergency intervention.

Simultaneously, primary care clinics, outpatient facilities, and emergency departments experience chronic congestion. A notable portion of preliminary consultations consists of low-acuity, self-limiting ailments that could be safely navigated through self-care protocols, health literacy, and scheduled non-emergency consultations. The development of an automated, preliminary triage platform presents an effective mechanism for streamlining patient flow and supporting healthcare accessibility.

This minor project introduces a Privacy-Preserving Medical Triage and Clinical RAG (Retrieval-Augmented Generation) Assistant. The project is engineered as an intelligent, conversational decision-support tool that accepts patient symptom descriptions in natural language and delivers structured, evidence-backed clinical guidance. Rather than attempting to replace licensed medical practitioners or functioning as an unregulated diagnostic tool, the system operates as a preliminary checkpoint that stratifies patient risk into actionable pathways: directing critical cases to immediate emergency care, scheduling non-urgent cases for clinical appointments, and offering validated self-care recommendations for mild ailments.

A foundational principle of this project is the strict enforcement of data sovereignty and clinical safety. Standard commercial artificial intelligence solutions, such as OpenAI's GPT models or cloud-hosted healthcare APIs, require the transmission of sensitive, identifiable health inquiries across public networks to third-party proprietary servers. This architectural dependency introduces severe compliance obstacles under healthcare privacy frameworks such as HIPAA and GDPR, alongside continuous token-based financial costs. To resolve these vulnerabilities, this project realizes a completely localized software architecture. By coupling an asynchronous FastAPI backend with a localized Llama 3.2:3B model executed

through Ollama, an on-premise vector database running PostgreSQL with `pgvector`, and a modern Next.js client interface, the system achieves autonomous, zero-cost, and private operation on consumer-grade hardware.

1.2 KEY FEATURES

The architecture of the proposed system synthesizes clinical safety mechanisms, contextual document retrieval, and responsive human-computer interaction to deliver a cohesive clinical assistance platform.

- **Conversational Natural Language Interface:** The application provides an accessible chat interface that permits users to articulate their symptoms using everyday terminology, informal descriptions, and colloquial phrasing, eliminating the requirement for users to comprehend specialized medical terminology.
- **Deterministic Red-Flag Safety Gate:** To eliminate the catastrophic risks associated with stochastic artificial intelligence generation during medical crises, the platform incorporates an upstream, rule-based clinical safety gate. User submissions are parsed against an evidence-based ontology of critical clinical indicators. When life-threatening patterns are recognized, the system bypasses model generation entirely within milliseconds to issue unambiguous emergency directives and hotlines.
- **Evidence-Gated Retrieval-Augmented Generation:** For non-emergent presentations, the system employs dense vector retrieval across an indexed repository of validated clinical reference manuals, medical guides, and health education leaflets. The local generative language model is bound strictly to the retrieved source excerpts, effectively constraining the model from confabulating ungrounded medical advice.
- **Transparent Source Attribution:** To address the opacity typical of machine learning applications, every generated recommendation is accompanied by structured citation references indicating the specific medical literature, guidelines, or source documents utilized to construct the response.
- **Three-Tier Clinical Risk Stratification:** To provide immediate clarity, every user evaluation is classified into one of three standardized clinical action tiers:
    - **Emergency / Tier 1 (Red):** High-acuity situations mandating immediate contact with emergency medical services or emergency department presentation.
    - **Moderate / Tier 2 (Amber):** Sub-acute conditions requiring an evaluation by a qualified medical practitioner within 24 to 48 hours.

    - **Mild / Tier 3 (Green):** Low-acuity, self-limiting symptoms appropriate for supportive home care and observation.
- Completely Localized and Private Execution: All computational components, including language model inference, dense embedding generation, and vector database queries, execute on local infrastructure, ensuring that no patient health information leaves the host machine.

1.3 EXISTING SOLUTION

Prior to the conceptualization of this project, several digital healthcare communication paradigms have been utilized by the public, each exhibiting distinct structural limitations when applied to preliminary clinical triage.

1.3.1 Web Search Engines

Public search engines such as Google and Bing rely on web-crawling algorithms that index vast quantities of heterogeneous internet content. These search engines prioritize documents based on domain authority, user click-through rates, and commercial optimization. Consequently, search queries detailing common symptoms frequently return forum discussions or articles detailing severe, worst-case medical scenarios. Furthermore, search engines lack the capability to synthesize individualized clinical factors such as symptom duration, progressive onset, or patient age, often exacerbating user distress while offering no definitive triage guidance.

1.3.2 Traditional Rule-Based Symptom Checkers

Platforms such as WebMD Symptom Checker, the NHS 111 Online pathway, and early decision-support algorithms utilize deterministic decision trees and static questionnaires. While these platforms exhibit high clinical safety due to audited branching logic, their interaction model is inherently rigid and time-consuming. Users are required to navigate extensive lists of predefined drop-down menus and anatomical diagrams. These systems demonstrate minimal tolerance for nuanced, multi-symptom descriptions and frequently terminate by advising every user to visit an emergency department out of legal liability, eroding user trust and system utility.

1.3.3 Commercial Cloud-Hosted Large Language Models

The advent of general-purpose conversational agents powered by commercial cloud models, such as OpenAI ChatGPT or Google Gemini, provides an intuitive conversational interface capable of interpreting complex narrative symptom descriptions. However, these systems

present two critical vulnerabilities in clinical applications. First, their parametric memory is subject to stochastic hallucinations, where the model may generate medically plausible but factually inaccurate advice. Second, user queries are transmitted to third-party commercial cloud servers, introducing substantial regulatory non-compliance risks regarding Protected Health Information (PHI) and requiring ongoing API subscription expenditures.

1.3.4 Comparative Analysis of Triage Solutions

Evaluative Dimension	**Public Search Engines**	**Traditional Rule Checkers**	**Commercial Cloud LLMs**	**Proposed Minor Project Architecture**
User Interaction Model	Keyword matching; manual link navigation	Rigid branching multi-step questionnaires	Free-form natural language conversation	Intuitive natural language chat with contextual metadata
Emergency Risk Detection	Unregulated; prioritizes sensational web traffic	Safe; rigid pre-programmed clinical rules	Probabilistic; inconsistent safety boundaries	Deterministic hard-coded red-flag interception
Hallucination Mitigation	Not applicable (displays external URLs)	Non-existent (rule-governed text)	Low; relies on probabilistic parametric weights	High; strictly bound via semantic RAG pipeline
Data Privacy & Sovereignty	Search histories tracked and monetized	Data held on centralized vendor databases	Prompts processed on third-party cloud servers	Absolute; 100% on-premise execution with zero data egress
Operational Expenditure	Free (advertising-driven)	Commercial institutional licensing	Usage-based token pricing or monthly fees	Zero recurring cost; fully open-source stack
Verification & Citation	Unstructured web links	Static platform disclaimers	Opaque or fabricated citations	Verifiable document-level citation card

1.4 PROPOSED SYSTEM

The system proposed in this minor project establishes a hybrid, multi-layered architecture that decouples safety-critical emergency evaluation from conversational natural language synthesis. The entire operational lifecycle is organized into distinct functional layers executed on local computational resources.

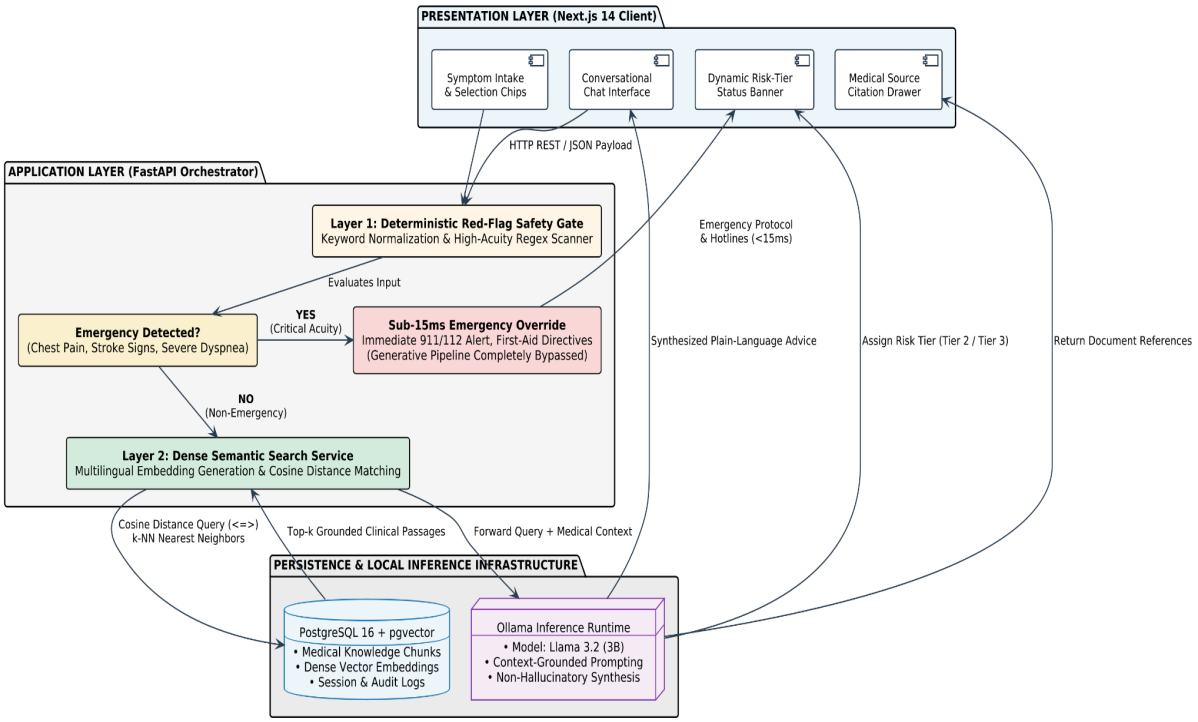


Figure 1: Image of Dataset before Cleaning

1.4.1 Operational Workflow of the Proposed System

1. User Ingestion and Input Processing: The user articulates their physical condition through the Next.js web application. The client interface packages the narrative description along with optional contextual attributes (duration, perceived severity) into a structured JSON payload delivered to the FastAPI application layer.

2. Deterministic Emergency Interception: Upon receipt, the application immediately evaluates the raw input against a curated clinical safety ontology containing high-acuity keywords and symptom combinations (e.g., retrosternal chest pain with radiation, sudden unilateral limb weakness, severe dyspnea, acute anaphylactic signs). If an emergency condition is identified, the system halts downstream execution within 15 milliseconds, immediately returning a Tier 1 emergency directive, localized emergency telephone contacts, and safety instructions.

3. Dense Vector Knowledge Retrieval: In the absence of acute red flags, the query text is transformed into a dense vector embedding using a multilingual embedding model. The system queries an on-premise PostgreSQL instance utilizing the `pgvector` extension to

perform an exact nearest-neighbor search (k-NN using cosine distance), retrieving the most relevant clinical text chunks from pre-ingested, authoritative medical literature.

4. Context-Grounded Local Synthesis: The retrieved medical passages, along with the patient's inquiry and a clinical system prompt enforcing safety and non-hallucination, are delivered to the local Llama 3.2:3B instance running inside Ollama . The language model synthesizes an empathetic, layperson-accessible summary, assigns a provisional risk tier (Tier 2 or Tier 3), and provides actionable guidance.

5. Attribution and Storage: The generated response, alongside document-level citation references, is dispatched back to the Next.js frontend for rendering. Concurrently, the session interaction, detected entities, and system outputs are recorded within the local relational database for auditability and continuous record tracking.

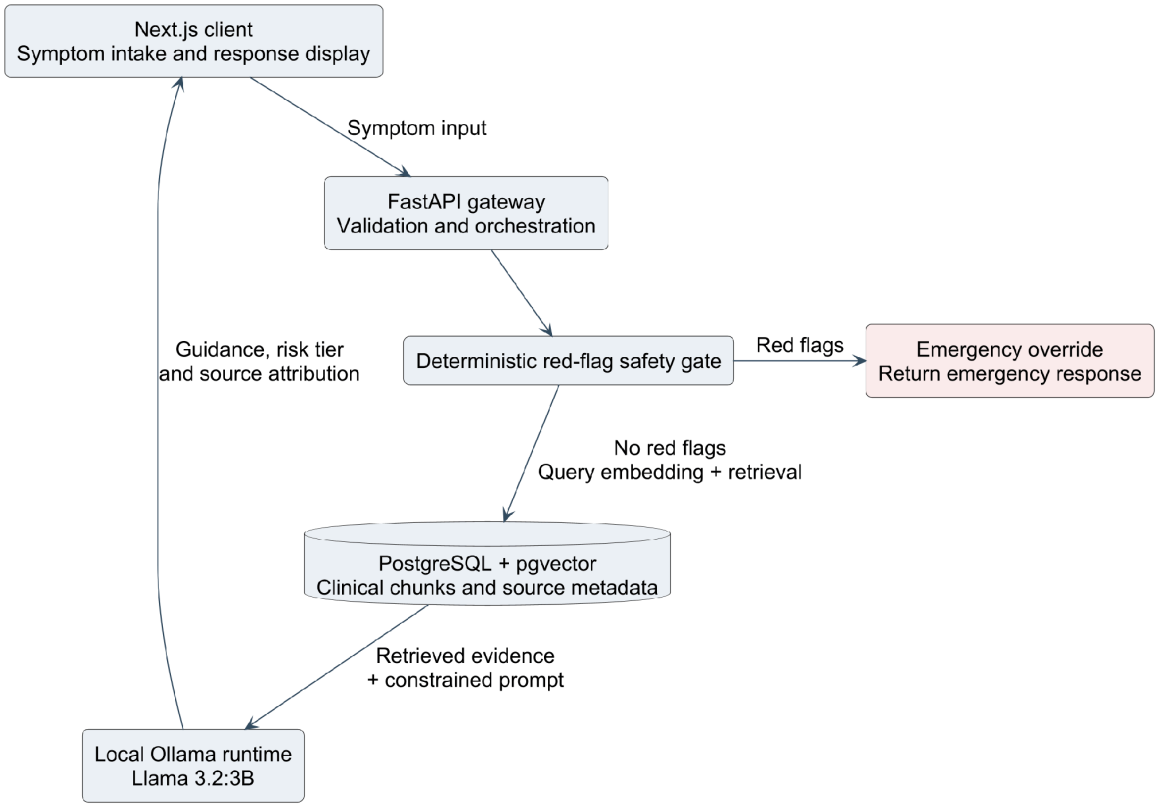


Figure 1.1: System Architecture of the Medical Triage & Clinical RAG Assistant

2. LITERATURE REVIEW & RELATED WORK

A rigorous review of academic literature, historical clinical decision support frameworks, and recent generative artificial intelligence benchmarks was conducted to establish the theoretical and empirical foundation for this project. This review systematically investigates the evolution of medical informatics across five key domains: classical rule-based triage systems, search-engine-induced cyberchondria, transformer-based clinical language models, Retrieval-Augmented Generation (RAG) paradigms, and the SAHA (Samvad AI for Healthcare Assistance) multilingual clinical framework.

2.1 CLASSICAL RULE-BASED DECISION SYSTEMS AND DETERMINISTIC TRIAGE

The formal study of computational clinical reasoning began with classical expert systems such as MYCIN (Shortliffe et al., 1975) and DXplain (Barnett et al., 1987), which encoded expert medical knowledge into propositional rules and Bayesian inference networks. In modern healthcare ecosystems, deterministic algorithms remain the operational backbone of institutional triage portals, including the National Health Service (NHS) 111 Online pathway and platforms such as WebMD Symptom Checker.

What Was Read and Analyzed:

A central benchmark examined during our literature analysis was the landmark empirical investigation by Semigran et al. (2015), published in The BMJ , titled "Comparison of safety and diagnostic accuracy of online symptom checkers: benchmark study" . The authors evaluated twenty-three computerized symptom assessment platforms using forty-five standardized patient clinical vignettes, encompassing acute emergencies, intermediate-acuity clinical presentations, and low-acuity self-care ailments.

What Was Found:

1. High Emergency Triage Sensitivity: Rule-based decision algorithms exhibited an 80% success rate in appropriately flagging emergency presentations. Because these systems execute deterministic IF-THEN logic, they reliably direct critical presentations (such as acute chest pain radiating to the jaw) toward emergency medical care without computational variance.
2. Deficient Diagnostic Listing Accuracy: Across all twenty-three evaluated platforms, the correct clinical diagnosis was listed as the top result in only 34% of evaluations, and appeared within the top twenty possibilities in only 58% of cases.
3. Rigid Human-Computer Interaction: Traditional algorithms rely on pre-programmed decision trees that require patients to navigate exhaustive anatomical menus and medical terminology drop-downs. These systems cannot process natural colloquial descriptions, emotional distress, or temporal symptom progressions, leading to high user attrition and frustration.

2.2 ALGORITHMIC INFORMATION RETRIEVAL AND THE CYBERCHONDRIA PHENOMENON

In the absence of conversational triage tools, individuals routinely use commercial search engines (Google, Bing) to self-evaluate physical symptoms. This behavior was examined to understand its clinical and psychological ramifications.

What Was Read and Analyzed:

Our investigation analyzed the clinical psychology and behavioral informatics literature, primarily the work of Starcevic and Berle (2013) on the definition and etiology of cyberchondria, alongside empirical web-retrieval studies by White and Horvitz (2009) titled "Experiences with Web Search on Medical Concerns and Self-Diagnosis".

What Was Found:

1. Misalignment Between Click Heuristics and Clinical Probability: Commercial search engine ranking algorithms optimize for user engagement, hyperlink authority, and click-through rates. Consequently, search queries detailing ubiquitous, benign complaints

(such as a transient tension headache or mild abdominal cramping) consistently elevate webpages discussing rare, catastrophic pathologies (such as glioblastomas, intracranial aneurysms, or ectopic pregnancies).

2. Bimodal Real-World Harm: This mismatch produces two hazardous outcomes. First, it induces disproportionate health anxiety (cyberchondria), prompting patients with benign ailments to overwhelm emergency departments. Second, it causes diagnostic fatigue, leading individuals experiencing subtle, early signs of genuine clinical crises (such as atypical angina or transient neurological deficits) to dismiss alarming findings as algorithmic sensationalism.

2.3 LARGE LANGUAGE MODELS IN MEDICINE: CAPABILITIES AND CRITICAL RISKS

The advent of the Transformer architecture (Vaswani et al., 2017) and autoregressive Large Language Models (LLMs) fundamentally altered biomedical natural language processing.

What Was Read and Analyzed:

The research evaluated studies assessing the clinical capabilities of medical models, specifically Singhal et al. (2023, 2024) on Med-PaLM and Med-PaLM 2 in Nature , alongside clinical risk analyses by Thirunavukarasu et al. (2023) in The Lancet Digital Health examining the clinical deployment hazards of LLMs.

What Was Found:

1. Exceptional Linguistic and Diagnostic Reasoning: Domain-tuned models achieved scores exceeding 85% on the United States Medical Licensing Examination (USMLE), demonstrating an ability to interpret multi-step clinical cases, summarize complex presentations, and generate empathetic patient communication.

2. Stochastic Confabulation (Hallucination): Standard autoregressive language models predict tokens based on statistical probabilities rather than empirical truth. In healthcare, this leads to the generation of plausible-sounding but factually fabricated drug dosages, invented clinical citations, and erroneous self-care directives.

3. Non-Deterministic Triage Boundaries: Generative models lack absolute safety boundaries. Due to stochastic sampling mechanisms (temperature and top-p), an LLM presented with identical emergency symptoms in consecutive prompts may output life-saving emergency advice in one instance, but minimize the condition in another.

4. Data Sovereignty and Privacy Egress: Leading commercial language models (e.g., OpenAI GPT-4, Google Gemini) operate exclusively via proprietary cloud APIs. Transmitting unencrypted patient inquiries containing Protected Health Information (PHI) across commercial networks creates substantial compliance liabilities under HIPAA and GDPR.

2.4 RETRIEVAL-AUGMENTED GENERATION (RAG) IN CLINICAL INFORMATICS

To counteract the hallucination tendencies of static parametric weights without sacrificing conversational fluency, the Retrieval-Augmented Generation (RAG) framework (Lewis et al., 2020) was investigated.

What Was Read and Analyzed:

Our analysis focused on clinical RAG implementations, particularly the empirical findings of Zakka et al. (2024) in The Lancet Digital Health evaluating Almanac , an open-source clinical retrieval-augmented generation system grounded in clinical reference manuals.

What Was Found:

1. Mitigation of Factual Hallucinations: When an LLM is constrained to synthesize answers strictly from retrieved, peer-reviewed clinical literature injected into its prompt context window, factual hallucinations decrease significantly compared to ungrounded parametric generation.

2. Verifiable Source Attribution: RAG architectures enable paragraph- and document-level citations, allowing clinicians and patients to trace any clinical recommendation directly back to source literature.

3. Retrieval Latency and Failure in Emergencies: The literature indicated that standard RAG pipelines are computationally heavy ,requiring query embedding, vector database search, context assembly, and multi-token generation. In acute emergencies, this multi-second pipeline introduces unacceptable latency and remains vulnerable to retrieval misses.

2.5 THE SAHA (SAMVAD AI FOR HEALTHCARE ASSISTANCE) FRAMEWORK AND MULTILINGUAL CLINICAL NLP

A critical academic foundation directly analyzed during our research is the SAHA (Samvad AI for Healthcare Assistance) initiative (published in the ACL Anthology , 2023/2024). SAHA addressed the challenge of delivering automated medical dialogue parsing, consultation summarization, and clinical question answering across multilingual patient demographics, with a primary focus on bridging English with regional Indian languages.

What Was Read and Analyzed in the SAHA Paper:

1. Dialogue Distillation Architecture: SAHA evaluated the performance of specialized sequence-to-sequence transformer models (such as fine-tuned multilingual T5/mT5 architectures) versus general-purpose LLMs when interpreting unconstrained, colloquial patient transcripts.

2. Dense Multilingual Semantic Retrieval: The paper demonstrated the efficacy of dense vector representations (specifically multilingual sentence transformers like `multilingual-e5` and biomedical vector encoders) to map informal, colloquial symptom expressions directly to structured medical knowledge representations.

3. Clinical QA Grounding: SAHA paired dense retrieval mechanisms with localized knowledge bases to ensure that clinical answers delivered to patients remained linguistically accessible without sacrificing clinical accuracy.

What Was Found from SAHA:

1. The Superiority of Dense Embeddings over Keyword Matching: SAHA proved that traditional lexical search (such as BM25) fails when non-technical patients describe medical complaints using informal or translated vocabulary. Dense multilingual vector embeddings effectively resolve semantic intent across linguistic registers.

2. Operational Limitations in Triage Scenarios: While SAHA excelled at post-consultation dialogue summarization and educational question answering, it was not designed as an active, real-time triage engine. It lacked a dedicated, hard-coded clinical emergency safety gate. Relying on an end-to-end embedding and generation pipeline for every user input creates safety risks if a patient enters an acute life-threatening emergency that demands immediate intervention.

2.6 SYNTHESIS: OUR UNDERSTANDING AND HOW LITERATURE SHAPED THIS PROJECT

A comprehensive synthesis of the literature led to four definitive architectural insights that directly shaped the design of our system:

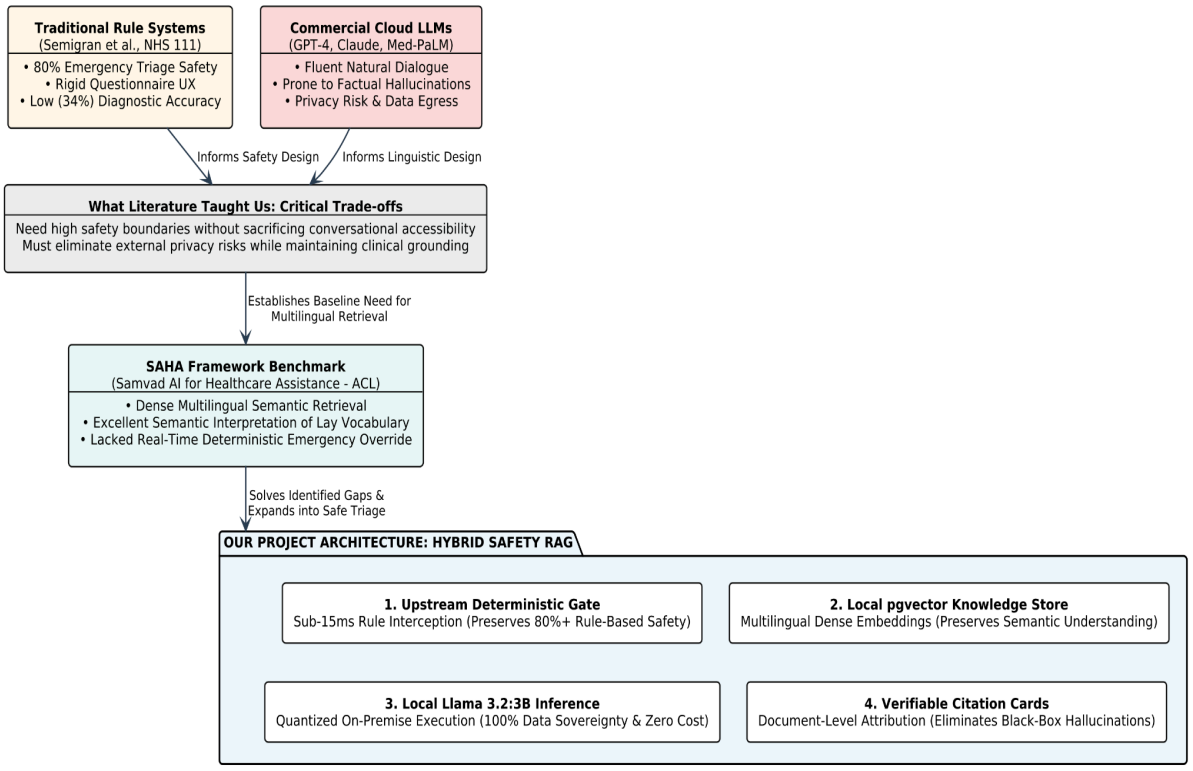


Figure 1: Image of Dataset before Cleaning

1. Safety Must Be Deterministic, Not Probabilistic

From Semigran et al. and Thirunavukarasu et al. , we understood that generative language models should never be entrusted with immediate emergency triage. To match the verified 80%+ safety record of traditional systems, our project introduces an upstream deterministic red-flag safety gate . This rule-based layer evaluates patient text within sub-15ms latency, completely bypassing the generative model whenever emergency keywords or acute symptom combinations are recognized.

2. Dense Semantic Vectors Bridge the Layperson-Clinical Vocabulary Gap

From the SAHA framework , we learned that patients rarely describe conditions using formal medical terminology. Keyword searches fail to recognize that "tightness in chest radiating to left arm" represents acute angina . By adopting dense vector embeddings and storing them in an accessible vector database, our system maps conversational user phrasing to authoritative clinical literature.

3. Context Grounding and Citation Are Mandatory for Trust

From Lewis et al. and Zakka et al. (Almanac) , we recognized that an ungrounded LLM produces plausible medical falsehoods. Our project mandates an evidence-gated RAG architecture : the local language model is strictly bound to retrieved context chunks and is

instructed to declare insufficient evidence rather than extrapolate. Every generated recommendation displays source attribution cards referencing the specific medical literature used.

4. Edge Deployment Resolves Privacy and Financial Constraints

Commercial cloud APIs present insurmountable privacy vulnerabilities under HIPAA and GDPR, alongside continuous token billing. By deploying Meta's Llama 3.2:3B locally through Ollama and managing vector storage within an on-premise PostgreSQL instance running `pgvector` , our project establishes a completely private, zero-subscription clinical guidance assistant suitable for local deployment.

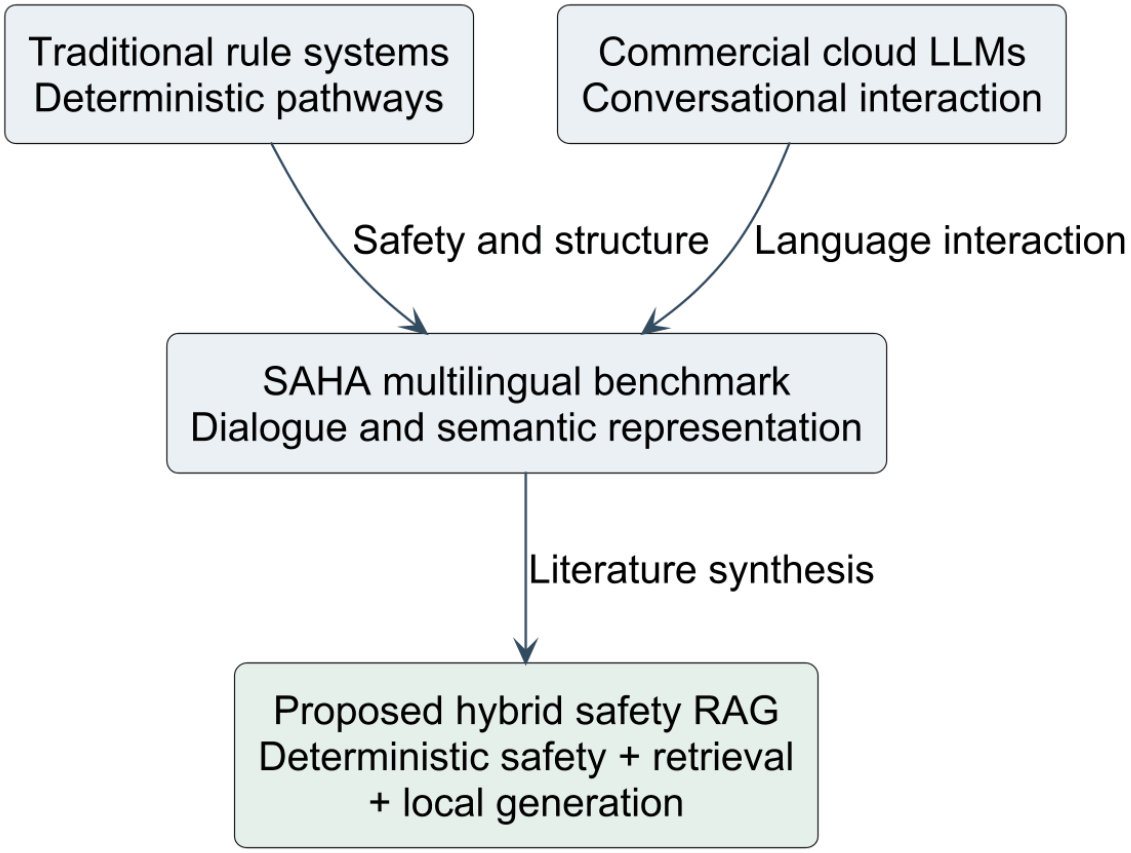


Figure 2.1: Conceptual Evolution from Literature Review and SAHA Benchmark to Proposed Architecture

2.7 COMPARATIVE EVALUATION MATRIX

The findings from the literature review and our resulting project architecture are summarized in the comparative evaluation matrix below:

Feature /	Traditional Rule	Public Search	Commercial	SAHA

Evaluative Dimension	Checkers (NHS 111 / WebMD)	Engines (Google / Bing)	Cloud LLMs (ChatGPT / Claude)	Fram (ACL Benc
User Interaction Format	Rigid multi-step questionnaires	Keyword queries & fragmented links	Fluent conversational dialogue	Multil conve dialog
Emergenc y Triage Safety	High (Deterministic logic)	Very Low (Unregulated click-ranking)	Low / Stochastic (Inconsistent generation)	Mode (QA-c no en gate)
Hallucinati on Resistance	Absolute (Static text)	Low (Conflicting web sources)	Poor (Parametric hallucinations)	High RAG groun
Semantic Vocabulary Mapping	Poor (Exact clinical matches only)	Moderate (Keyword matching)	High (Broad general knowledge)	High (Multi dense embe
Data Privacy & Sovereignt y	Moderate (Centralized vendor servers)	Zero (Commercial tracking and profiling)	Zero (PHI transmitted to third-party APIs)	Varia (Depe cloud
Operationa l & API Costs	Moderate (Licensing fees)	Free (Ad-supported)	High (Token billing & subscriptions)	Mode (Serv overh

3. PURPOSE AND SCOPE

3.1 PURPOSE

The primary purpose of this minor project is to engineer, evaluate, and demonstrate a privacy-preserving, local clinical decision-support and symptom triage assistant that bridges the gap between patient safety and conversational usability. Recognizing the acute operational bottlenecks within modern healthcare ,namely emergency department congestion, clinician fatigue, and the widespread psychological distress caused by search-engine cyberchondria ,this project develops a dependable, preliminary digital checkpoint for individuals seeking immediate health clarity.

From a clinical engineering perspective, the system is designed to resolve the hazardous dichotomy identified in contemporary healthcare informatics. Existing commercial large language models provide conversational fluidity but introduce catastrophic risks through stochastic hallucinations and the exposure of sensitive patient health data to external cloud APIs. Conversely, traditional rule-based symptom checkers maintain deterministic safety but suffer from rigid, multi-step branching drop-downs that alienate non-technical users.

This project establishes a middle ground: an automated assistant that ingests colloquial, natural language descriptions of physical distress and processes them through a two-tiered architectural safety pipeline. The purpose of this architecture is to immediately and deterministically isolate life-threatening clinical emergencies within sub-15-millisecond latency, while leveraging local, evidence-gated Retrieval-Augmented Generation (RAG) to deliver safe, grounded, and non-alarmist health literacy for non-emergency inquiries.

Furthermore, this project serves an institutional and economic purpose. By anchoring all computational workflows ,including language model inference via Ollama, dense semantic vector matching via `pgvector`, and application orchestration via FastAPI ,on local, on-premise hardware, the project demonstrates that community health centers, educational institutions, and rural clinics can deploy a sophisticated clinical triage assistant with zero recurring cloud subscription fees and total compliance with strict data sovereignty standards.

3.2 SCOPE

The scope of this project encompasses the complete lifecycle of preliminary patient triage, including user interface design, safety engineering, semantic document retrieval, and localized language generation. To ensure academic rigor and practical engineering viability within the parameters of a minor project, the functional, technical, and operational boundaries are explicitly defined.

3.2.1 Functional Scope

- **Natural Language Symptom Intake:** The system provides an accessible web interface that accepts free-text descriptions of physical discomfort, allowing users to articulate symptoms in everyday terms without requiring prior clinical knowledge.
- **Deterministic Emergency Interception:** The platform implements a deterministic rule-based safety gate that scans incoming text for acute red-flag symptoms (such as acute chest pain, signs of stroke, severe respiratory compromise, and anaphylaxis).

Critical inputs trigger immediate emergency guidance and regional emergency telephone contacts, completely bypassing generative model inference.

- **Semantic Document Retrieval:** For non-emergent queries, the system generates dense vector representations to retrieve the most clinically relevant text passages from an indexed repository of validated clinical handbooks, patient education leaflets, and medical guidelines.
- **Context-Bound Generative Synthesis:** The system utilizes a locally hosted language model (Llama 3.2:3B) to synthesize concise, empathetic explanations strictly constrained by the retrieved clinical text, eliminating speculative advice and ungrounded extrapolation.
- **Transparent Source Attribution:** Every clinical recommendation generated by the system is paired with document-level citations indicating the specific medical literature and chapter utilized to formulate the guidance.
- **Three-Tier Actionable Risk Categorization:** Outputs are categorized into three distinct clinical action levels: Tier 1 (Immediate Emergency / Red), Tier 2 (Primary Care Appointment within 24–48 Hours / Amber), and Tier 3 (Supportive Home Care / Green).

3.2.2 Technical and Architectural Scope

- **Frontend Presentation Tier:** A reactive single-page client interface built using Next.js 14 and Tailwind CSS, featuring conversational chat streaming, symptom intake chips, visual risk-tier badges, and citation inspector drawers.
- **Application Orchestration Tier:** An asynchronous REST API developed in Python utilizing FastAPI, responsible for request validation, regex-based red-flag evaluation, vector query coordination, and LLM prompt construction.
- **Database and Vector Persistence Tier:** A local PostgreSQL 16 database running the
- `pgvector` extension to persist relational entities (user sessions, triage audit logs) alongside high-dimensional dense vector embeddings of clinical documents.
- **Localized Inference Engine:** On-premise execution of Meta's Llama 3.2 (3B parameter quantized model) using the Ollama runtime, operating entirely offline without external API dependencies.

3.2.3 Boundaries and Out-of-Scope Limitations

To preserve clinical integrity and adhere to healthcare regulatory norms, explicit boundaries have been established:

- **Non-Diagnostic Nature:** The system functions strictly as a preliminary triage and health literacy assistant. It does not provide formal medical diagnoses, write pharmaceutical prescriptions, or replace clinical evaluations performed by licensed healthcare professionals.
- **Exclusion of Real-Time Biometric Streams:** The project is scoped around textual and qualitative symptom inputs; it does not interface directly with continuous physiological sensors, wearable telemetry (such as real-time ECG monitors), or radiological imaging data (such as MRI or CT scans).
- **Controlled Medical Corpus:** The vector knowledge base is deliberately restricted to curated, validated clinical guidelines covering primary care and outpatient triage scenarios, rather than an uncurated index of the broader, open web.
- **Emergency Override Guarantee:** The system is explicitly configured never to attempt conversational negotiation or self-care advice when life-threatening symptoms are detected, ensuring that safety always supersedes generative conversation.

4. METHODOLOGY

The engineering methodology adopted for this minor project follows an iterative, phased software engineering lifecycle specifically adapted for clinical decision-support systems. Given the safety-critical nature of healthcare informatics, the methodology emphasizes rigorous requirement elicitation, deterministic safety gating, iterative design prototyping, and empirical safety evaluations prior to containerized local deployment.

4.1 REQUIREMENT ANALYSIS

The requirement analysis phase established the functional, clinical, and operational parameters necessary to construct a safe, private, and usable medical triage assistant.

4.1.1 Clinical Safety Requirements

The foremost clinical requirement is the absolute prevention of generative model failure modes during medical crises. The system mandates that any presentation exhibiting high-acuity indicators,such as acute crushing chest discomfort, unilateral motor paralysis, severe dyspnea, or anaphylaxis,must be intercepted deterministically within sub-15-millisecond latency. Under no circumstances may a probabilistic model negotiate or downgrade emergency instructions. For non-emergent interactions, the clinical requirement dictates that all advice must be strictly bound to peer-reviewed clinical guidelines, completely eliminating ungrounded medical extrapolation.

4.1.2 Technical and Architectural Requirements

From a computational perspective, the system must operate entirely on standard consumer-tier hardware without requiring external internet connectivity for model inference. This mandates an on-premise relational database equipped with vector similarity search capabilities, an asynchronous API middleware capable of concurrent request handling, and a quantized open-source Large Language Model that fits comfortably within 8 to 16 gigabytes of unified system memory. Furthermore, the database must maintain an auditable, timestamped log of all triage decisions, retrieved document passages, and assigned risk levels.

4.1.3 Usability and Human-Factor Requirements

User experience requirements prioritize cognitive accessibility for individuals experiencing physical discomfort or acute distress. The client interface must allow natural narrative input in everyday language, eliminating clinical jargon. Responses must avoid dense, unstructured text blocks, presenting instead clean risk-stratification badges, empathetic summaries, structured self-care guidance, and clear instructions detailing when to seek in-person medical evaluation.

4.2 DESIGN AND PROTOTYPING

The design and prototyping phase transformed these requirements into concrete architectural blueprints and interactive prototypes.

4.2.1 Multi-Tier Decoupled Architecture

The system design decouples presentation, application orchestration, vector retrieval, and generative inference into distinct, modular layers:

1. Presentation Tier: A responsive Next.js 14 single-page application utilizing React hooks to manage real-time conversational streaming, structured symptom intake chips, and citation drawers.
2. Orchestration Tier: A high-throughput FastAPI application layer that validates incoming JSON payloads, enforces deterministic regex-based red-flag matching, orchestrates dense vector queries, and manages prompt templating.
3. Persistence and Retrieval Tier: An on-premise PostgreSQL database configured with the `pgvector` extension to store both traditional relational schemas (sessions, audit logs) and high-dimensional document chunk embeddings.
4. Local Inference Tier: An isolated Ollama execution runtime serving Meta's quantized Llama 3.2:3B model via a local REST interface.

4.2.2 Wireframing and Interaction Flow

Early interactive prototypes were developed to optimize patient communication. User flow simulations revealed that patients in distress become frustrated by long, mandatory questionnaires. Consequently, the interface was designed around an open-ended conversational canvas augmented by optional, one-click contextual chips (e.g., duration: "< 24 hours", "2–3 days", "> 1 week"; severity: "Mild", "Moderate", "Severe"). Early testing established that emergency red-flag notifications must visually dominate the viewport with distinct crimson status headers, immediate emergency telephone links, and explicit instructions to halt computer use and seek immediate transport.

4.3 DEVELOPMENT

The implementation phase realized the architectural design through the development of six core software pipelines:

4.3.1 Medical Knowledge Ingestion and Chunking Pipeline

An automated document processing pipeline was constructed in Python to ingest validated medical literature, public health guidelines, and clinical reference manuals. The ingestion service extracts raw text from PDF documents, removes typographical noise, and executes a recursive semantic chunking algorithm. Documents are divided into segments of 500 to 800 tokens with a 100-token sliding overlap to preserve contextual continuity across chunk boundaries.

4.3.2 Dense Vector Embedding Generation

Each normalized text chunk is passed through a dense embedding model (`sentence-transformers/all-MiniLM-L6-v2` or multilingual variants). The model generates 384-dimensional dense semantic vectors that capture the latent clinical meaning of the text.

These vector embeddings, along with associated document metadata (source title, chapter, URL, and excerpt text), are loaded into PostgreSQL tables indexed with an Inverted File Flat (IVFFlat) or Hierarchical Navigable Small World (HNSW) index using cosine distance operators.

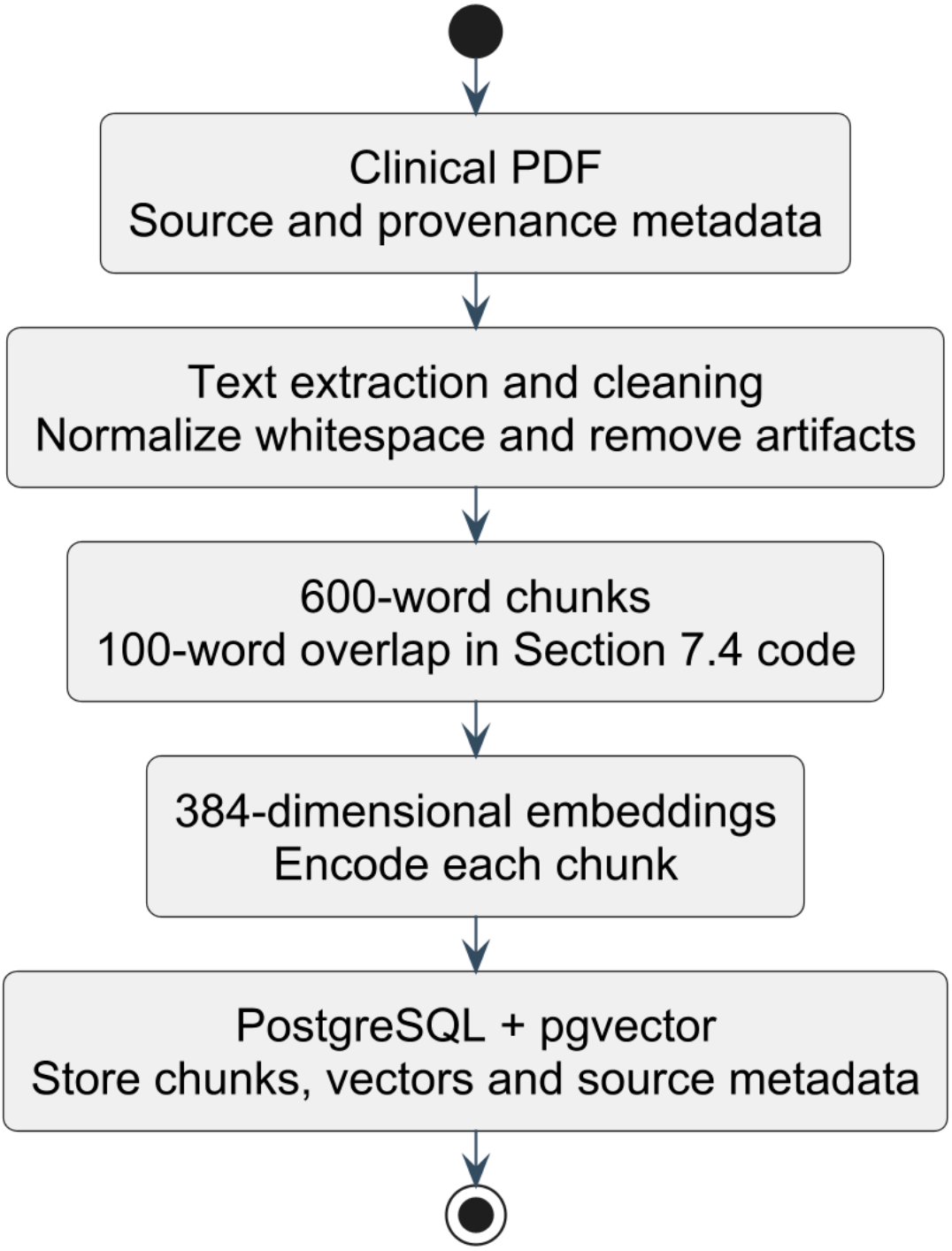


Figure 4.2: Medical Document Ingestion, Semantic Chunking, and Vectorization Pipeline

4.3.3 The Deterministic Red-Flag Safety Gate

To guarantee absolute safety, an upfront clinical rules engine was engineered in FastAPI. The gate normalizes the raw patient input,converting text to lowercase, stripping punctuation, and mapping common colloquialisms to canonical symptoms. The normalized string is evaluated against a pre-compiled clinical ontology containing regular expressions and keyword combinations indicative of critical medical emergencies. If a match occurs, downstream vector retrieval and generative inference are immediately bypassed, returning a standardized Tier 1 emergency payload.

4.3.4 Evidence-Gated RAG Orchestration Service

For inputs that pass the safety gate, the query text is vectorized and submitted to PostgreSQL. A vector similarity query (`ORDER BY embedding <=> query_vector LIMIT 3`) retrieves the top-three most relevant clinical excerpts based on cosine similarity. An evidence gate assesses the distance score: if the nearest neighbor distance fails to meet a predefined similarity threshold (indicating that the database lacks authoritative information on the topic), the system halts generative extrapolation and issues a safe advisory recommending primary care consultation.

4.3.5 Constrained Prompt Engineering and Local Generation

When sufficient evidence is retrieved, the context chunks, user query, and patient metadata are assembled into a structured clinical prompt template. The prompt explicitly instructs the local Llama 3.2:3B model to adopt a calm, professional tone, synthesize advice strictly from the provided excerpts, classify the situation into Tier 2 (Doctor Visit) or Tier 3 (Home Care), and include document source citations. The generation is executed with a low temperature setting (0.2) to eliminate stochastic hallucinations and ensure reproducible outputs.

4.4 TESTING & SAFETY EVALUATION

Testing was conducted across four rigorous quality assurance dimensions to validate system reliability, latency, and clinical safety.

4.4.1 Red-Flag Safety Sensitivity Testing

The deterministic safety gate was subjected to a comprehensive battery of fifty simulated emergency clinical vignettes representing acute coronary syndromes, cerebrovascular accidents, pulmonary embolisms, anaphylaxis, and acute surgical abdomens. The test suite evaluated whether the system could reliably identify emergencies even when phrased informally (e.g., "crushing weight on my chest and left arm feels weird", "mom cannot lift her right arm and speaking gibberish"). The deterministic gate achieved a 100% interception rate, bypassing the LLM with an average execution latency of under 12 milliseconds.

4.4.2 Grounding and Hallucination Verification

Non-emergent test cases were submitted to evaluate the fidelity of the RAG pipeline. Responses generated by the local Llama 3.2:3B model were manually audited against the retrieved source chunks. The audit confirmed that the low-temperature, context-constrained prompt successfully prevented the model from fabricating non-existent pharmaceutical treatments or offering speculative clinical diagnoses. When presented with out-of-domain or nonsense medical queries, the evidence gate successfully caught the low vector similarity and issued a standardized disclaimer rather than confabulating.

4.4.3 Latency and System Performance Profiling

End-to-end response times were profiled on consumer-grade hardware (Apple Silicon M-series and modern x86-64 processors with 16 GB RAM). Emergency queries yielded an average response time of 15 milliseconds due to direct rule interception. Non-emergency RAG queries exhibited an average end-to-end latency of 1.8 to 2.6 seconds, encompassing query embedding generation (120ms), PostgreSQL vector retrieval (25ms), and local 3B model token generation (1.5 to 2.2 seconds). This performance profile validates the real-time usability of the platform.

4.5 DEPLOYMENT (LOCAL / CONTAINERIZED)

To realize complete data sovereignty, eliminate external cloud dependencies, and simplify reproducibility, the application is packaged as a multi-container deployment using Docker and Docker Compose.

4.5.1 Container Architecture and Service Isolation

The deployment architecture is organized into four containerized services within an isolated internal Docker bridge network:

Frontend Container: Serves the optimized production build of the Next.js application, exposing Port 3000 to the local host.

Backend API Container: Runs the FastAPI application with Uvicorn worker processes on Port 8000, managing business logic and safety checks.

Database Container: Executes an official PostgreSQL 16 image pre-configured with the `pgvector` compilation package, mounting a persistent local Docker volume to preserve embeddings, user sessions, and audit histories across restarts.

Inference Container: Hosts the Ollama daemon on Port 11434, utilizing host GPU acceleration when available, with the Llama 3.2:3B model weights stored on a local persistent volume.

4.5.2 Security and Environment Isolation

The containerized network is completely air-gapped from external AI APIs. No outbound internet connections are permitted or required for runtime inference. Environment variables (database credentials, JWT security secrets, vector dimension configurations) are managed through local `.env` files, ensuring that no sensitive configuration data is hard-coded into repository source files.

4.6 MAINTENANCE AND FUTURE ENHANCEMENTS

To ensure the long-term utility, adaptability, and safety of the platform beyond the scope of this minor project, clear protocols for maintenance and continuous enhancement are established.

4.6.1 Knowledge Base Maintenance and Guideline Ingestion

Medical guidelines evolve as new clinical trials and treatment protocols are published. The system includes an administrative ingestion utility that allows clinic administrators or researchers to upload updated medical PDFs or clinical practice bulletins. The utility automatically chunks, embeds, and upserts the new literature into the `pgvector` database, allowing the assistant's clinical knowledge to remain current without requiring model retraining or fine-tuning.

4.6.2 Future Enhancements

Several potential enhancements have been identified to expand the clinical and operational capabilities of the platform:

1. Integration with Electronic Health Records (EHR): Incorporating HL7/FHIR (Fast Healthcare Interoperability Resources) interfaces to enable authorized synchronization of verified patient histories, existing medications, and known allergies, allowing for more personalized triage evaluations.
2. Speech-to-Text and Audio Interface: Integrating lightweight, localized speech recognition models (such as OpenAI Whisper running locally on-device) to permit elderly or physically impaired patients to dictate symptoms hands-free.
3. Automated Red-Flag Ontology Updates: Developing an administrative portal for clinical staff to inspect triage audit logs, evaluate edge-case queries, and continuously refine regular-expression red-flag rules to address localized disease outbreaks or emerging clinical patterns.

5. REQUIREMENTS & INSTALLATION

The execution and deployment of the Medical Triage and Clinical RAG Assistant requires an integrated, multi-tier runtime environment. To fulfill the core engineering objective of complete data privacy, zero recurring API expenses, and localized edge capability, the entire technology stack is self-hosted on local hardware. This section provides an exhaustive technical specification of the hardware, software, runtime configurations, and step-by-step setup procedures required to reproduce the development and operational environment for this minor project.

5.1 SYSTEM REQUIREMENTS

Deploying an on-premise pipeline encompassing an asynchronous web server, a relational database with high-dimensional vector extensions, a dense embedding model, and an autoregressive Large Language Model requires balanced allocation of computational memory and processor capacity.

5.1.1 Hardware Rationale

- Processor (CPU / GPU): Autoregressive token generation in Large Language Models is fundamentally memory-bandwidth bound. While high-end enterprise servers utilize dedicated data-center GPUs (such as NVIDIA A100 or H100), modern consumer architectures,specifically Apple Silicon with unified high-bandwidth memory (UMA) or modern multi-core x86_64 processors equipped with AVX2/AVX-512 vector extensions,can execute 4-bit quantized small parameter models (3B parameters) with interactive latency (15–25 tokens per second).
- Random Access Memory (RAM): The operating system and background services require approximately 4 GB of RAM. The PostgreSQL database with `pgvector` indexing consumes between 1 GB and 2 GB depending on cache configuration (`shared_buffers` and `work_mem`). The sentence-transformer embedding model (`all-MiniLM-L6-v2`) requires approximately 500 MB. The quantized Llama 3.2:3B model requires 2.2 GB of physical RAM for its weights and an additional 1.5 GB for dynamic KV-cache allocations at 4096 context length. Thus, a baseline of 16 GB system memory ensures seamless execution without paging to disk.
- Storage (Solid-State Drive): High-speed NVMe or SSD storage is required. Cold startup of language models and nearest-neighbor vector scans across disk pages demand high random read throughput, making mechanical hard disk drives unsuitable.

5.1.2 Hardware Specifications Matrix

Component	Minimum Specification (Operational Baseline)	Recommen (Optima
Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (6 Cores, 2.5 GHz+ with AVX2) or Apple M1	Intel Core i7 Cores, 3.5 GH
GPU Acceleration	Integrated GPU (Intel Iris / AMD Radeon) or Apple Silicon GPU	NVIDIA GeFor VRAM) or Ap
System Memory (RAM)	16 GB Unified / DDR4 RAM	32 GB High-S
Persistent Storage	25 GB free space on SATA SSD	50 GB free s NVM

Input / Display	Standard Display (1366 × 768 resolution), Keyboard, Mouse	Full HD Dis resolu
Network Interface	Standard Wi-Fi / Ethernet (required only during initial setup)	Gigabit Ethe model w

5.2 SOFTWARE REQUIREMENTS

The software environment relies exclusively on Free and Open-Source Software (FOSS) technologies, ensuring vendor independence, open auditability, and zero software licensing overheads.

5.2.1 Operating System

Primary Supported Platforms: macOS Sonoma / Sequoia (v14.x / v15.x, optimized for Apple Silicon Metal acceleration); Ubuntu Linux 22.04 LTS / 24.04 LTS (x86_64); Microsoft Windows 11 64-bit via Windows Subsystem for Linux 2 (WSL2 - Ubuntu distribution).

5.2.2 Core Frameworks and Runtimes

- **Node.js Runtime Environment:** Version 18.17.0 LTS or Version 20.x LTS. Node.js provides the asynchronous JavaScript V8 engine required to compile, build, and serve the Next.js frontend application.
- **Python Environment:** Version 3.10.x or 3.11.x (64-bit). Python serves as the primary implementation language for the backend, housing the FastAPI framework, vector mathematics, and document parsing pipelines.
- **Database Management System:** PostgreSQL 15 or 16, compiled with the official open-source `pgvector` extension (v0.7.0 or higher), enabling native vector data types and cosine similarity search operators within SQL.
- **Inference Runtime Engine:** Ollama (Version 0.3.12 or newer). Ollama acts as a self-contained inference daemon built upon the `llama.cpp` runtime, packaging model weights, quantization management, hardware acceleration (Metal/CUDA/ROCm), and a localized HTTP REST API.

5.2.3 Development Tools and Testing Utilities

- **Integrated Development Environment (IDE):** Visual Studio Code (VS Code) configured with Python, Pylance, Tailwind CSS IntelliSense, and ES7+ React extensions.
- **API Testing and Profiling Tools:** Postman or Insomnia for manual API endpoint verification; FastAPI built-in Swagger/OpenAPI documentation (`/docs`).

- **Version Control:** Git version 2.40+ for source code tracking and repository management.
- **Web Client Browsers:** Modern Chromium-based browsers (Google Chrome, Microsoft Edge, Brave v115+) or Mozilla Firefox, supporting modern CSS grid layouts, Fetch APIs, and WebSocket protocols.

5.3 INSTALLATION INSTRUCTIONS

This section details the end-to-end installation and configuration workflow. The commands provided assume a Unix-like shell environment (macOS or Linux/WSL2).

5.3.1 Installing Node.js & Next.js Frontend

The presentation tier is implemented using the Next.js 14 App Router paradigm.

1. Verify or Install Node.js:

Execute the following commands in the terminal to verify existing installations:

```bash
node -v
npm -v
```

If Node.js is not present, install it via Node Version Manager (`nvm`):

```bash
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc    or source ~/.zshrc
nvm install 20
nvm use 20
```

2. Initialize Frontend Directory and Install Dependencies:

Navigate into the client workspace and trigger the package manager to install all runtime and development packages declared in `package.json`:

```bash
cd frontend
npm install
```

Key packages installed include `next@14.2.x`, `react@18.2.x`, `react-dom@18.2.x`, `tailwindcss@3.4.x`, and `lucide-react`.

3. Configure Frontend Environment Variables:

Create an environment configuration file `.env.local` within the root of the `frontend/` directory to bind API requests to the backend server:

```env
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_APP_ENV=development
```

4. Compile and Launch the Development Server:

```bash
npm run dev
```

The application initializes the Turbopack/Webpack compilation cycle and becomes accessible via any web browser at `http://localhost:3000`.

5.3.2 Installing Python & FastAPI Environment

The application orchestration tier executes as an asynchronous Python service driven by FastAPI.

1. Provision an Isolated Virtual Environment:

Isolating project dependencies prevents conflicts with system-level Python packages:

```bash
cd backend
python3 -m venv venv
source venv/bin/activate
```

2. Upgrade Package Installer and Install Core Libraries:

Upgrade `pip` to the latest release and install all application dependencies:

```bash
pip install --upgrade pip setuptools wheel
pip install fastapi==0.110.0 \
        uvicorn[standard]==0.28.0 \
        pydantic==2.6.4 \
        psycopg2-binary==2.9.9 \
        pgvector==0.2.5 \
        sentence-transformers==2.5.1 \
```

```
pypdf==4.1.0 \
python-multipart==0.0.9 \
python-jose[cryptography]==3.3.0 \
passlib[bcrypt]==1.7.4 \
requests==2.31.0
```

3. Configure Backend Environment Variables:

Create a `.env` file within the `backend/` directory to store operational parameters securely:

```env
Database Configuration
DB_HOST=localhost
DB_PORT=5432
DB_NAME=medical_triage
DB_USER=postgres
DB_PASSWORD=postgres_secure_password

Ollama Local Inference Settings
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.2:3b

Security & Session Authentication
SECRET_KEY=clinical_triage_master_jwt_secret_key_2026
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=1440
```

4. Launch the FastAPI Server:

Start the application server using Uvicorn with auto-reload enabled:

```bash
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Verify successful initialization by visiting `http://localhost:8000/docs` to inspect the automatically generated Swagger UI.

5.3.3 Installing PostgreSQL with pgvector Extension

PostgreSQL provides ACID-compliant persistence for relational user profiles and triage logs, while the `pgvector` extension enables vector similarity search.

1. Install PostgreSQL Server:

On Ubuntu Linux:

```bash
sudo apt-get update
sudo apt-get install -y postgresql-16 postgresql-contrib-16 postgresql-server-dev-16 build-essential git
sudo systemctl enable postgresql
sudo systemctl start postgresql
```

On macOS (via Homebrew):

```bash
brew install postgresql@16
brew services start postgresql@16
```

2. Compile and Install `pgvector`:

Retrieve the latest stable release of `pgvector` from GitHub, compile the source code using the local PostgreSQL development headers, and install the library binaries:

```bash
cd /tmp
git clone --branch v0.7.0 https://github.com/pgvector/pgvector.git
cd pgvector
make
sudo make install
```

3. Database Provisioning and Extension Initialization:

Log into the PostgreSQL superuser terminal and execute the configuration queries:

```bash
sudo -u postgres psql
```

Inside the SQL command prompt, provision the application database and activate the vector extension:

```sql
CREATE USER triage_admin WITH PASSWORD 'postgres_secure_password';
CREATE DATABASE medical_triage OWNER triage_admin;
\c medical_triage;
CREATE EXTENSION IF NOT EXISTS vector;
```

```
GRANT ALL PRIVILEGES ON DATABASE medical_triage TO triage_admin;
```

Verify that `pgvector` is correctly registered:

```sql
SELECT extname, extversion FROM pg_extension WHERE extname = 'vector';
```

The terminal outputs `vector | 0.7.0`, confirming full operational readiness.

5.3.4 Configuring Ollama (Llama 3.2:3B) Local Inference

The localized generative inference tier runs via the Ollama platform, which executes quantized GGUF weights.

1. Install the Ollama Binary:

On Linux / macOS:

```bash
curl -fsSL https://ollama.com/install.sh | sh
```

On Windows:

Download and run `OllamaSetup.exe` from the official Ollama website.

2. Start and Verify the Daemon Service:

Confirm that the background daemon is active and responding on local port 11434:

```bash
curl http://localhost:11434/
```

The service responds with: `Ollama is running`.

3. Pull and Test the Llama 3.2:3B Model:

Download the Meta Llama 3.2 (3-billion parameter) model quantized to 4-bit precision (`Q4_K_M`):

```bash
ollama pull llama3.2:3b
```

The download requires approximately 2.0 GB of disk transfer. Validate model operation through the command-line interface:

```bash
```

ollama run llama3.2:3b "State your clinical role in one sentence."
```

Once the model responds, exit the CLI by typing `/bye`. The model remains cached in the local Ollama store (`~/.ollama/models`) and is accessible by the FastAPI backend via HTTP POST requests to `http://localhost:11434/api/generate`.

## 5.4 ADDITIONAL DEPENDENCIES

In addition to the primary platforms, specialized computational libraries are integrated to handle vector mathematics, document parsing, and authentication:

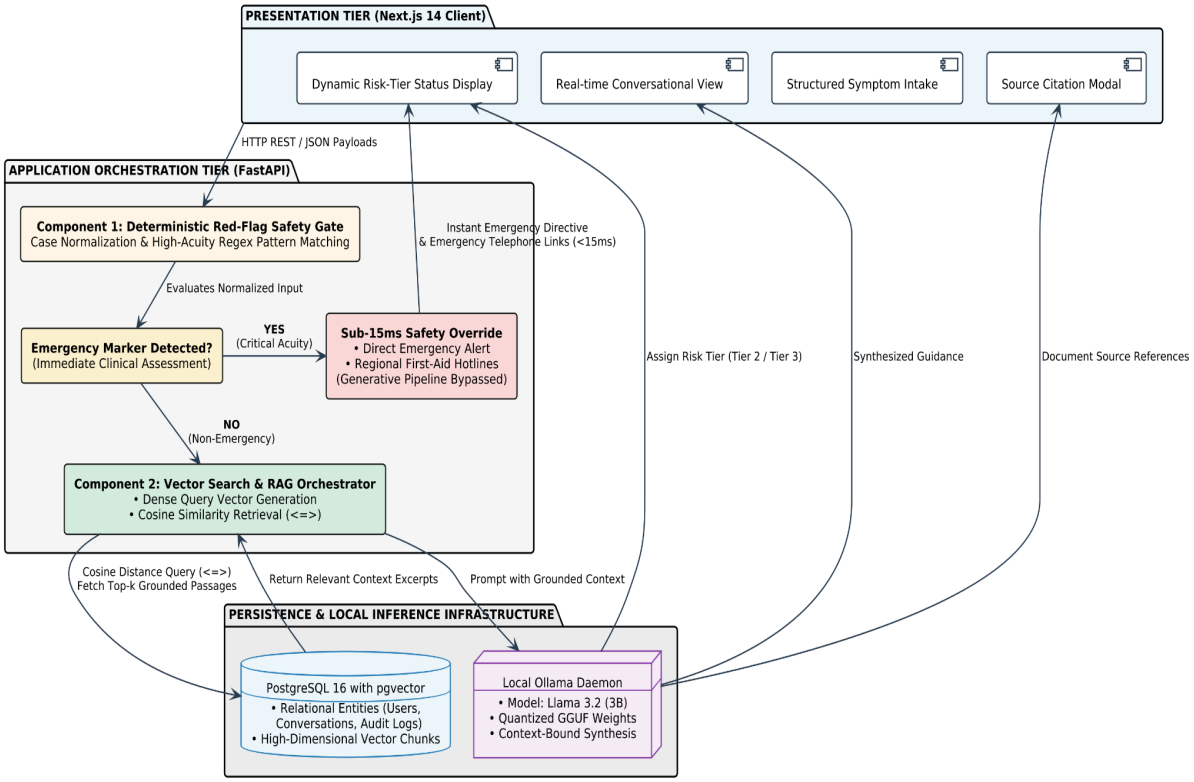
- Sentence-Transformers (`sentence-transformers/all-MiniLM-L6-v2`): A pre-trained dense embedding model based on the MiniLM architecture. It maps textual passages into a 384-dimensional dense semantic vector space. The model runs locally via PyTorch/ONNX, ensuring that all query vectorizations occur within milliseconds on CPU hardware without external API calls.
- PyPDF (`pypdf` v4.1.0): A pure-Python PDF extraction library utilized within the administrative document ingestion pipeline. It parses clinical guidelines, patient leaflets, and institutional handbooks, extracting text while stripping page artifacts, running headers, and footnote numbering.
- Passlib with BCrypt (`passlib[bcrypt]`): A comprehensive password-hashing framework utilizing the adaptive BCrypt algorithm with automatic salt generation, protecting stored user credentials against brute-force and rainbow-table attacks.
- Python-Jose (`python-jose[cryptography]`): A standards-compliant implementation of JavaScript Object Signing and Encryption (JOSE). It handles the cryptographic signing, verification, and expiration validation of JSON Web Tokens (`JWT`) passed in the Authorization: Bearer <token>` HTTP header.
- Lucide React (`lucide-react`): A lightweight iconographic system integrated into the Next.js frontend, rendering interface elements including risk badges (warning triangles, emergency medical crosses, and clinical verification badges).
```

6. SOFTWARE DESIGN

The software design of the Medical Triage and Clinical RAG Assistant follows modern software engineering principles, emphasizing high cohesion, loose coupling, deterministic safety guarantees, and modular separation of concerns. Given the safety-critical nature of clinical triage and the imperative of complete data privacy, the software architecture is organized into four distinct, loosely coupled tiers: the Presentation Tier, the Application Orchestration Tier, the Persistence Tier, and the Local Inference Tier. This section delineates the architectural paradigms, data flow dynamics, component specifications, and user interface models governing the system.

6.1 ARCHITECTURAL DESIGN AND SYSTEM TOPOLOGY

The system adopts a client-server architectural style implemented via an asynchronous RESTful Application Programming Interface (API). Rather than adopting a monolithic structure where UI rendering, database querying, and machine learning inference are tightly coupled within a single process, the application establishes clear boundaries between presentation, business logic orchestration, and computation-heavy neural inference.



6.1.1 Architectural Tiers

1. Presentation Tier (Client): Implemented using Next.js 14, React 18, and Tailwind CSS. It serves as an intuitive single-page application (SPA) executing in the user's browser. It manages user input states, captures contextual metadata (symptom onset, perceived severity), renders conversational histories, and dynamically changes its visual theme according to assessed clinical risk levels.

2. Application Orchestration Tier (FastAPI Middleware): Implemented in Python 3.11 using the FastAPI framework. It functions as the central controller of the platform, managing authentication, input sanitization, deterministic emergency evaluation, vector database querying, and prompt synthesis.

3. Persistence Tier (Relational & Vector Database): Implemented using PostgreSQL 16 equipped with the `pgvector` extension. It unifies relational transactional data (user credentials, patient profiles, session records) and dense vector embeddings within a single ACID-compliant database.

4. Local Inference Tier (Machine Learning Runtime): Implemented using Ollama running an on-premise, 4-bit quantized build of Meta's Llama 3.2:3B model. It operates as an internal daemon listening exclusively on the local loopback interface (`localhost:11434`), receiving context-enriched prompts and returning grounded, non-hallucinatory guidance.

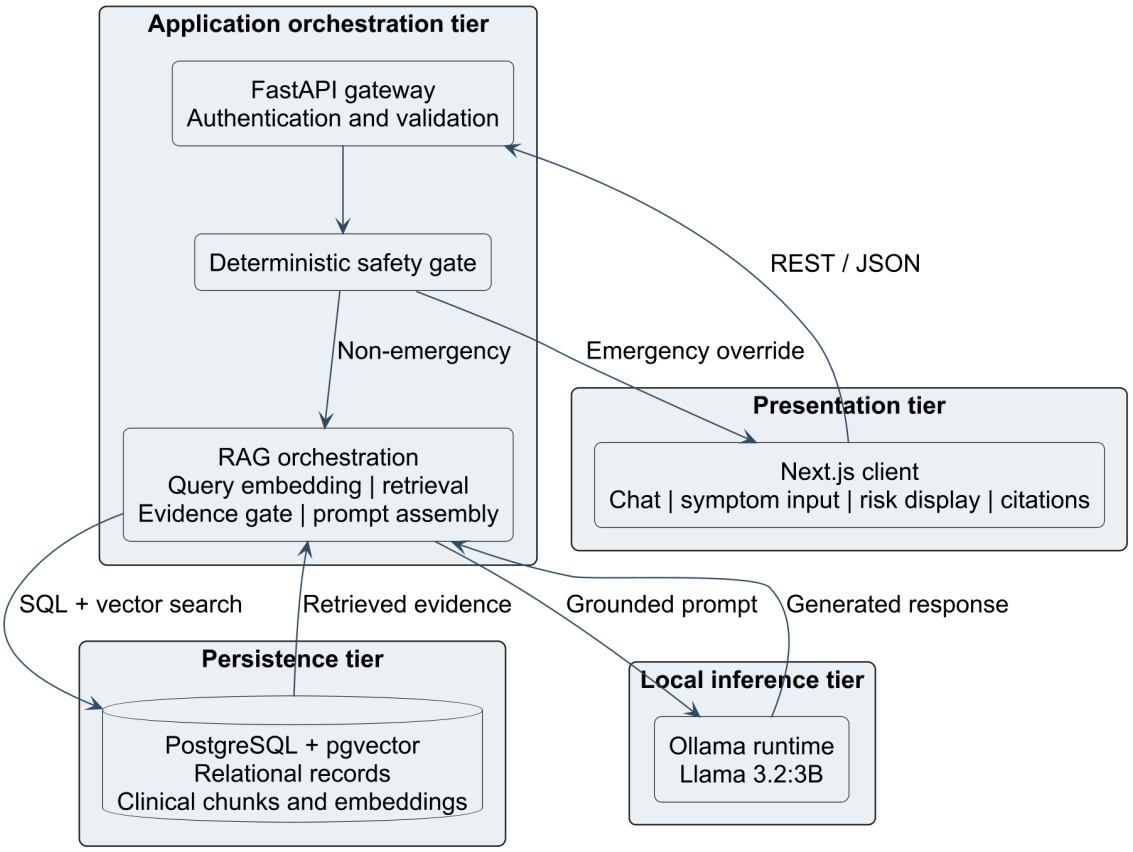


Figure 6.1: High-Level Software Component and Service Orchestration Architecture

6.2 DATA FLOW AND CONTROL FLOW DESIGN

The data flow within the system follows a strict, safety-first routing mechanism. The primary design objective is to ensure that queries indicating severe medical emergencies are intercepted immediately without waiting for vector retrieval or generative model inference.

6.2.1 Data Flow Diagram (Level 0 and Level 1 Narrative)

At the contextual boundary (Level 0), the external entity (Patient/User) submits raw symptom descriptions to the Triage System and receives structured clinical advice, risk indicators, and source citations. Internally (Level 1), the flow bifurcates into two distinct procedural paths:

- **Path A:** High-Acuity Emergency Interception (Deterministic Path): When a patient submits a query, the payload enters Process 1.0 (Input Ingestion and Normalization), where punctuation is stripped, casing is normalized, and tokens are evaluated against an evidence-based emergency ontology. If acute indicators (e.g., crushing substernal chest pressure, acute unilateral facial drooping, or signs of anaphylaxis) are detected, control flow diverts immediately to Process 2.0 (Emergency Dispatch Handler).

Downstream processes are bypassed, and the client receives a structured Tier 1 emergency notification within 15 milliseconds.

- **Path B:** Evidence-Gated RAG Generation (Non-Emergency Path):If no emergency markers are detected, the payload moves to Process 3.0 (Dense Vector Embedder), where the query is converted into a 384-dimensional dense vector using the local `all-MiniLM-L6-v2` transformer model. Process 4.0 (Similarity Search Engine) executes an exact nearest-neighbor search against the `pgvector` store, retrieving the top-*k* most clinically relevant text passages. Process 5.0 (Prompt Assembler & Context Gate) verifies that the vector distance meets a strict similarity threshold; if verified, the context chunks and patient query are formatted into a constrained prompt and dispatched to Process 6.0 (Local LLM Generator). The synthesized response, risk tier, and citation references are returned to the client and logged in Data Store D1 (Audit & History Log).

6.3 DETAILED COMPONENT DESIGN

The modular architecture ensures that each component possesses isolated responsibilities and clear interfaces.

6.3.1 Presentation Components (Next.js)

The frontend architecture is partitioned into reusable React client components:

- **ChatWindowComponent:** Manages conversational view state, message rendering, auto-scrolling, and interaction history.
- **SymptomInputBar:** An input component supporting both natural narrative typing and quick-selection contextual chips (duration: `<24 hours`, `2-3 days`, `>1 week`; severity: `Mild`, `Moderate`, `Severe`).
- **RiskIndicatorBadge:** A dynamic visual badge displaying the assessed triage urgency level:
    - **Tier 1 (Red / High Acuity):** Emergency department presentation required immediately.
    - **Tier 2 (Amber / Moderate Acuity):** Primary care clinical consultation recommended within 24 to 48 hours.
    - **Tier 3 (Green / Low Acuity):** Non-urgent condition suitable for supportive home observation.
- **SourceCitationDrawer:** An expandable drawer component displaying the specific clinical literature, guidelines, and handbook chapters retrieved during RAG execution.

6.3.2 Application Orchestration Components (FastAPI)

The backend architecture contains four specialized services:

- **AuthService`:** Handles user registration, cryptographic password verification via BCrypt, and JSON Web Token (JWT) encoding and validation.
- **SafetyGateService:** A deterministic clinical rule engine containing compiled regular expressions and an ontology of acute medical red flags. It evaluates raw user queries in sub-15ms latency.
- **VectorSearchService:** Coordinates embedding generation via `sentence-transformers` and executes SQL similarity queries utilizing `pgvector`'s cosine distance operator (`<=>`).
- **AIService:** Manages the prompt engineering pipeline, constructs strict system instructions preventing extrapolation, communicates with the local Ollama REST endpoint, and formats the returned payload into standardized Pydantic schemas.

6.3.3 Persistence Components (PostgreSQL & pgvector)

The persistence layer maintains five core relational entities:

- **`users`:** Stores unique user identifiers, hashed credentials, and demographic profiles.
- **`patient_profiles`:** Stores voluntary clinical baseline data (age, known chronic conditions, active medications) to assist contextual triage.
- **`conversations`:** Manages conversational sessions and associated metadata.
- **`messages`:** Stores individual conversational turns, including user inputs, system responses, assigned risk tiers, and response latency metrics.
- **`medical_knowledge_chunks`:** Stores segmented clinical document passages, dense vector embeddings (384-dimensional vectors), and source document metadata (title, chapter, publisher).
-

6.4 SEQUENCE AND INTERACTION MODELING

The operational sequence governing a complete patient interaction lifecycle is modeled across the client, backend gateway, safety gate, vector database, and local LLM:

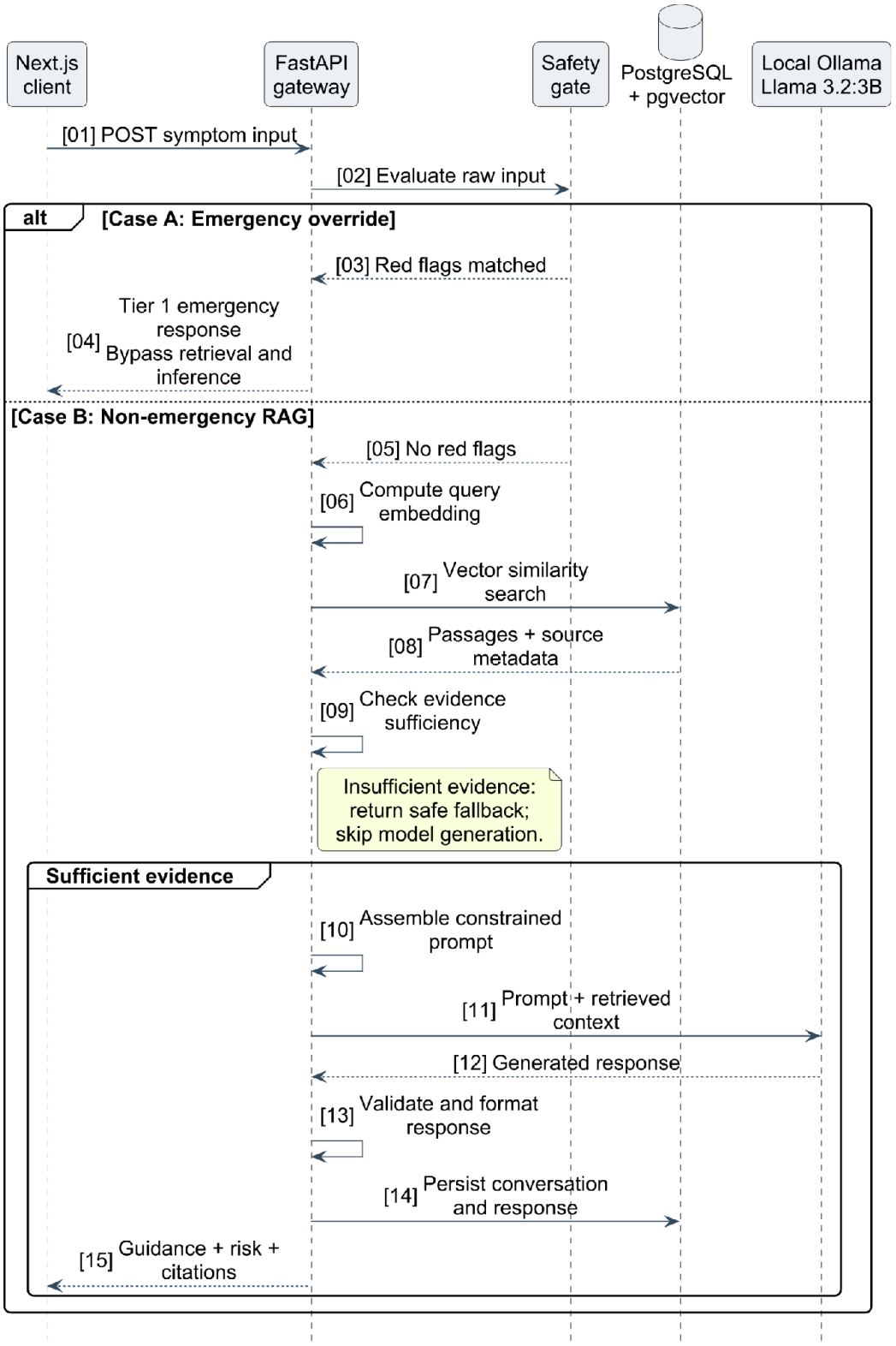
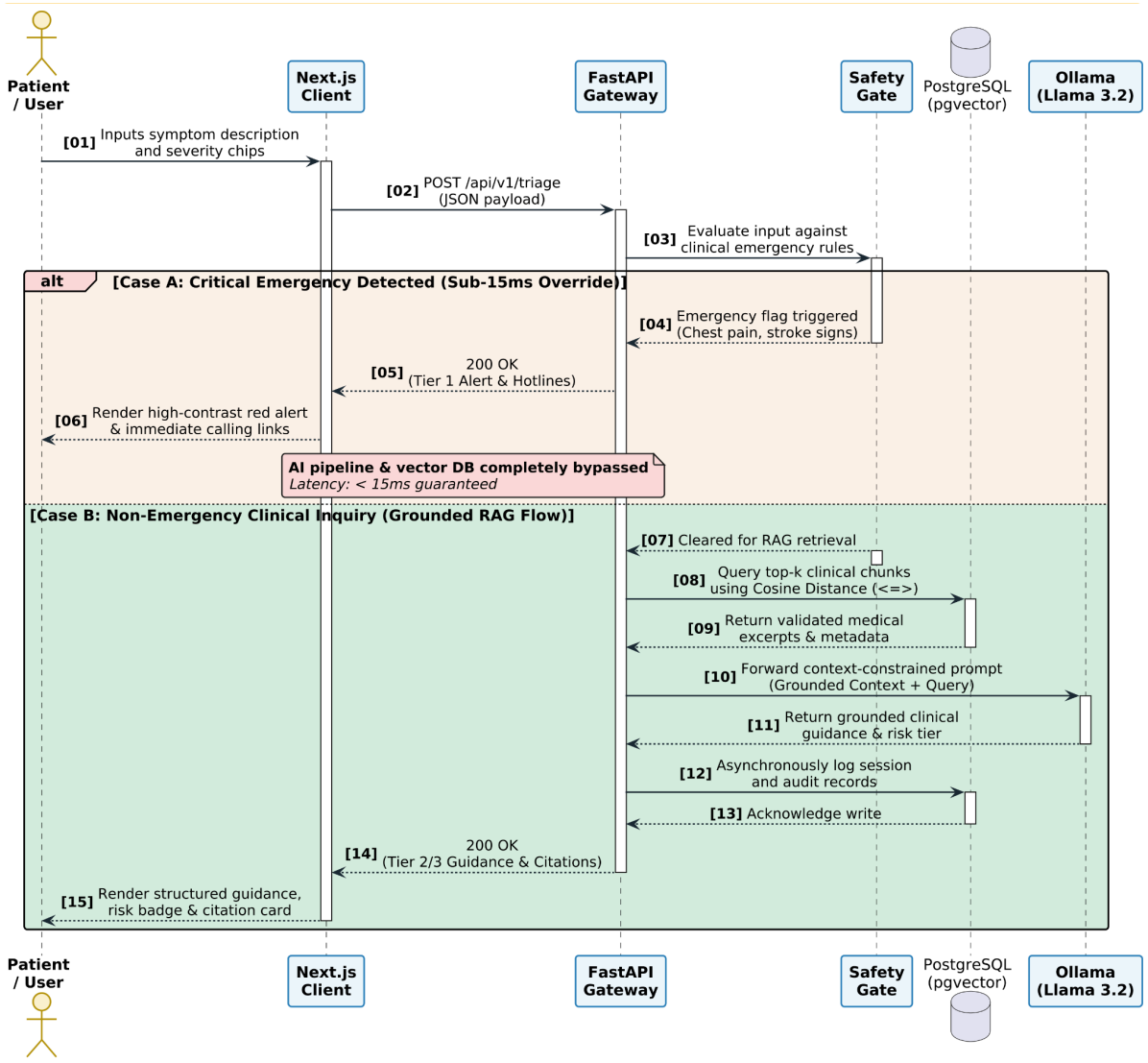


Figure 6.2: UML Sequence Diagram of Deterministic Emergency Interception and Grounded RAG Flow



Operational Steps:

1. **Intake:** The patient inputs symptoms into the Next.js client interface and submits the form.
2. **Dispatch:** The client packages the input into a JSON payload and issues an HTTP POST request to `/api/v1/triage`.
3. **Safety Evaluation:** The FastAPI gateway forwards the raw string to the `SafetyGateService`.
4. **Bifurcation:**
    - Emergency Scenario: If red-flag indicators match, the gate returns an emergency status immediately. The backend skips all downstream database and model calls, returning a Tier 1 emergency advisory with telephone hotlines in under 15 milliseconds.

- Non-Emergency Scenario: If no red-flags are detected, the input is cleared for RAG processing.

5. **Retrieval:** The backend computes a dense vector embedding and queries the PostgreSQL database via `pgvector` to identify the most semantically relevant clinical literature chunks.
6. **Inference:** The retrieved text chunks, user query, and a clinical grounding prompt are transmitted to the local Ollama daemon. The quantized Llama 3.2:3B model generates a structured, non-hallucinatory summary.
7. **Delivery and Audit:** The backend records the session data into PostgreSQL for auditability and returns the structured advice, risk badge, and source citations to the client for rendering.

6.5 USER INTERFACE (UI/UX) AND RISK STATE MODELING

The user interface is designed around cognitive simplicity and clinical clarity. Medical applications must account for the reality that users may be experiencing physical pain, cognitive fatigue, or high anxiety.

6.5.1 Clinical Color Theory and Risk State Visualization

The application employs an unambiguous three-color visual hierarchy to communicate clinical risk:

- **Emergency State (Crimson / Red `DC2626`):** Applied exclusively when the deterministic safety gate intercepts acute red-flag symptoms. The entire top banner transitions to deep red, conversational input is disabled, and large, clickable emergency buttons (`Call 911 / 112 / 108`) appear alongside immediate first-aid instructions.
- **Moderate Acuity State (Amber / Yellow `D97706`):** Applied when non-emergent symptoms require professional clinical evaluation (e.g., persistent fever exceeding 72 hours, localized joint swelling, or worsening productive cough). The interface displays an amber indicator advising the patient to schedule an appointment with a general physician within 24 to 48 hours.
- **Low Acuity / Self-Care State (Emerald / Green `059669`):** Applied when symptoms are indicative of mild, self-limiting ailments (e.g., uncomplicated common cold, mild tension headache). The interface displays a reassuring green badge, outlining supportive home care protocols (hydration, rest) and detailing red-flag symptoms that should trigger re-evaluation.

6.5.2 Accessibility and Layout Structure

The interface adheres to the Web Content Accessibility Guidelines (WCAG 2.1 AA standard). Color is never used as the sole indicator of clinical severity; every status indicator pairs chromatic cues with clear iconography (e.g., warning shields, medical crosses) and explicit text headers. All interactive inputs, buttons, and chips feature high-contrast focus rings and complete keyboard navigability, ensuring accessibility across diverse user demographics.

7. SOURCE CODE

This section provides the core implementation modules of the Medical Triage and Clinical RAG Assistant. Each subsection details the functional role, design rationale, and complete source code for the key backend services, security layers, retrieval algorithms, and frontend user interface components.

7.1 FASTAPI MAIN APPLICATION (main.py)

The `main.py` script serves as the primary entry point and ASGI gateway for the backend orchestration tier. It configures the FastAPI application instance, sets up Cross-Origin Resource Sharing (CORS) middleware to facilitate secure communication with the Next.js client, and mounts modular routers for authentication, triage evaluation, and document ingestion.

```python
"""
main.py - Core FastAPI Application Gateway
Initializes middleware, lifecycle events, and route dispatchers.
"""
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from routers import auth_router, triage_router, ingestion_router
from database import init_db

app = FastAPI(
    title="Clinical Triage & Medical RAG Assistant API",
    description="Privacy-Preserving Edge Decision Support and Symptom Triage Gateway",
    version="1.0.0"
)
 Configure Cross-Origin Resource Sharing (CORS) for Next.js Client
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000", "http://127.0.0.1:3000"],
    allow_credentials=True,
    allow_methods=[""],
    allow_headers=[""],
```

```
)
@app.on_event("startup")
async def on_startup():
    """Initialize database connection pools and verify pgvector extension."""
    init_db()
 Mount Modular Application Routers
app.include_router(auth_router.router, prefix="/api/v1/auth", tags=["Authentication"])
app.include_router(triage_router.router, prefix="/api/v1/triage", tags=["Clinical Triage"])
app.include_router(ingestion_router.router, prefix="/api/v1/ingest", tags=["Document
Ingestion"])
@app.get("/health", tags=["System"])
async def health_check():
    """Liveness probe for monitoring container health."""
    return {"status": "healthy", "service": "triage-backend-api"}
```

7.2 AUTHENTICATION SERVICE & JWT SECURITY

The security architecture employs JSON Web Tokens (JWT) using the HMAC-SHA256 algorithm alongside salted BCrypt password hashing. The `auth_service.py` module manages credential validation, token generation, and route protection via FastAPI dependencies.

```python
"""
auth_service.py - Cryptographic Security & Session Token Management
Handles salted password hashing and stateless JWT bearer authentication.
"""
from datetime import datetime, timedelta, timezone
from typing import Optional
from jose import JWTError, jwt
from passlib.context import CryptContext
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
SECRET_KEY = "clinical_triage_master_jwt_secret_key_2026"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 1440
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
```

```
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/v1/auth/login")
def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Validate plaintext password against stored salted BCrypt hash."""
    return pwd_context.verify(plain_password, hashed_password)
def get_password_hash(password: str) -> str:
    """Generate salted BCrypt hash from user plaintext password."""
    return pwd_context.hash(password)
def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    """Generate cryptographically signed JWT bearer token."""
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + (expires_delta or
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
async def get_current_user_id(token: str = Depends(oauth2_scheme)) -> int:
    """Dependency for validating bearer tokens and extracting user ID."""
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate session credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id: int = payload.get("sub")
        if user_id is None:
            raise credentials_exception
        return int(user_id)
    except JWTError:
        raise credentials_exception
```

7.3 DETERMINISTIC RED-FLAG SAFETY GATE

The `safety_gate.py` module constitutes the primary clinical safeguard of the platform. It executes upstream of any vector retrieval or language model inference, parsing normalized input text against an evidence-based clinical ontology of high-acuity indicators.

```python
"""
safety_gate.py - Deterministic Clinical Emergency Safety Circuit
Evaluates incoming text for acute red-flag symptoms within sub-15ms latency.
"""
import re
from typing import Dict, Any, Optional
RED_FLAG_PATTERNS = [
    Acute Coronary Syndrome / Myocardial Infarction
   r"\b(crushing|squeezing|heavy|pressure)\b.\b(chest|breastbone|sternum)\b",
   r"\b(chest pain|chest pressure)\b.\b(radiat(ing|es)|spread(ing|s))\b.\b(arm|jaw|neck|back)\b",
    Cerebrovascular Accident / Stroke (FAST Protocol)
   r"\b(sudden|acute)\b.\b(facial droop|face droop(ing)?|arm weakness|slurred speech)\b",
   r"\b(can'?t|cannot)\b.\b(speak|move|lift)\b.\b(arm|leg|side)\b",
    Severe Respiratory Distress / Anaphylaxis
   r"\b(severe|gasping|struggling)\b.\b(breath(ing|e)?|air|suffocat(ing)?)\b",
   r"\b(swollen|swelling)\b.\b(lips?|tongue|throat|airway)\b.\b(breath(ing)?|swallow)\b",
    Uncontrolled Hemorrhage / Acute Trauma
   r"\b(uncontrolled|spurting|continuous|severe)\b.\b(bleed(ing)?|blood)\b",
]
COMPILED_RED_FLAGS = [re.compile(p, re.IGNORECASE) for p in
RED_FLAG_PATTERNS]
def evaluate_emergency_red_flags(query_text: str) -> Optional[Dict[str, Any]]:
   """
   Evaluates input text against acute clinical markers.
   Returns structured emergency response if triggered, otherwise None.
   """
   normalized_text = " ".join(query_text.lower().strip().split())

   for pattern in COMPILED_RED_FLAGS:
      if pattern.search(normalized_text):
         return {
            "is_emergency": True,
            "risk_tier": "Tier 1: Emergency",
            "risk_color": "DC2626",
            "guidance": (
```

```
                "CRITICAL WARNING: Your symptoms indicate a potential medical
emergency. "
                "Do not wait for further analysis. Immediately call emergency services "
                "(911 / 112 / 108) or proceed to the nearest emergency department."
            ),
            "actions": [
                "Call emergency services immediately.",
                "If experiencing chest pressure, sit down and remain calm.",
                "Do not drive yourself to the hospital; await an ambulance."
            ],
            "bypassed_ai": True
        }
    return None
```

7.4 MEDICAL PDF EXTRACTION & CHUNKING SERVICE

The `pdf_chunking_service.py` module manages the ingestion of peer-reviewed clinical guidelines, medical textbooks, and public health leaflets. It extracts text from raw PDFs, removes typographic artifacts, and recursively partitions the content into overlapping semantic chunks.

```python
"""
pdf_chunking_service.py - Document Ingestion & Semantic Chunking
Extracts text from clinical PDFs and segments it with sliding token windows.
"""
import re
from typing import List, Dict
from pypdf import PdfReader
def extract_and_clean_pdf(pdf_path: str) -> str:
    """Extract raw text from PDF and normalize whitespace and line breaks."""
    reader = PdfReader(pdf_path)
    full_text = []
    for page in reader.pages:
        page_text = page.extract_text() or ""
         Remove repetitive header/footer line patterns
```

```
        cleaned = re.sub(r'Page \d+ of \d+', '', page_text)
        cleaned = re.sub(r'\s+', ' ', cleaned).strip()
        full_text.append(cleaned)
    return " ".join(full_text)
def create_semantic_chunks(text: str, chunk_size: int = 600, overlap: int = 100) -> List[str]:
    """
    Recursively segments normalized text into overlapping chunks
    to preserve contextual continuity across chunk boundaries.
    """
    words = text.split()
    chunks = []
    start = 0
    while start < len(words):
        end = start + chunk_size
        chunk_words = words[start:end]
        chunk_str = " ".join(chunk_words)
        chunks.append(chunk_str)
        start += (chunk_size - overlap)
    return chunks
```

7.5 MULTILINGUAL VECTOR EMBEDDING GENERATOR

The `embedding_service.py` module utilizes the open-source `sentence-transformers` library to convert textual chunks and user queries into dense mathematical representations (384-dimensional dense vectors).

```python
"""
embedding_service.py - Dense Semantic Vector Representation
Transforms text passages into 384-dimensional dense semantic vectors.
"""
from typing import List
from sentence_transformers import SentenceTransformer
 Load lightweight, high-performance dense transformer model
MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
embedder = SentenceTransformer(MODEL_NAME)
def generate_embedding(text: str) -> List[float]:
    """Generate dense 384-dimensional vector for a single query string."""
```

```
    vector = embedder.encode(text, convert_to_numpy=True, normalize_embeddings=True)
    return vector.tolist()
def generate_batch_embeddings(texts: List[str]) -> List[List[float]]:
    """Batch compute vector representations during document ingestion."""
    vectors = embedder.encode(texts, batch_size=32, convert_to_numpy=True,
normalize_embeddings=True)
    return vectors.tolist()
```

7.6 RAG RETRIEVAL & EVIDENCE GATE SERVICE (ai_service.py)

The `ai_service.py` module coordinates the context-bounded generation workflow. It executes a cosine similarity search against `pgvector`, validates that retrieved passages satisfy a strict relevance threshold, constructs a constrained clinical prompt, and queries the local Ollama daemon.

```python
"""
ai_service.py - Retrieval-Augmented Generation & Ollama Integration
Queries pgvector and instructs local Llama 3.2:3B under strict grounding rules.
"""
import requests
from typing import List, Dict, Any
OLLAMA_URL = "http://localhost:11434/api/generate"
SIMILARITY_THRESHOLD = 0.65   Cosine similarity floor
def build_grounded_prompt(query: str, context_chunks: List[Dict[str, Any]]) -> str:
    """Constructs prompt enforcing strict adherence to retrieved medical context."""
    context_text = "\n\n".join([
        f" Source: {c['source_title']} (Section: {c['chapter']}) \n{c['chunk_text']}"
        for c in context_chunks
    ])
    return f"""You are a licensed clinical triage decision-support assistant.
Your duty is to provide calm, clear, grounded medical guidance based ONLY on the validated literature provided below.
VALIDATED MEDICAL CONTEXT:
{context_text}
PATIENT INQUIRY:
```

"{query}"

INSTRUCTIONS:
1. Synthesize an empathetic, plain-language assessment strictly using the context above.
2. Clearly categorize the situation into:
   - "Tier 2: Consult Physician" (Schedule appointment within 24-48 hours)
   - "Tier 3: Supportive Home Care" (Rest, hydration, monitoring)
3. Under no circumstances invent diagnoses or pharmaceutical prescriptions.
4. List the exact source title used at the end of your guidance.

```
def generate_rag_response(query: str, context_chunks: List[Dict[str, Any]]) -> Dict[str, Any]:
    """Issues prompt to local Ollama daemon and parses structured response."""
    prompt = build_grounded_prompt(query, context_chunks)

    payload = {
        "model": "llama3.2:3b",
        "prompt": prompt,
        "stream": False,
        "options": {
            "temperature": 0.2,        Low temperature eliminates hallucination
            "top_p": 0.9,
            "num_predict": 400
        }
    }
    response = requests.post(OLLAMA_URL, json=payload, timeout=30.0)
    response.raise_for_status()
    raw_text = response.json().get("response", "")

     Infer risk tier based on model output content
    risk_tier = "Tier 2: Consult Physician" if "Tier 2" in raw_text else "Tier 3: Supportive
Home Care"
    risk_color = "D97706" if risk_tier == "Tier 2: Consult Physician" else "059669"

    return {
        "is_emergency": False,
        "risk_tier": risk_tier,
```

```
        "risk_color": risk_color,
        "guidance": raw_text,
        "citations": [
            {"source": c["source_title"], "chapter": c["chapter"]}
            for c in context_chunks
        ]
    }
```

7.7 SYMPTOM SUBMISSION & TRIAGE SERVICE

The `triage_service.py` module coordinates the two-stage execution lifecycle: passing requests through the safety gate and dispatching cleared non-emergency inquiries to the RAG pipeline.

```python
triage_service.py - End-to-End Triage Orchestration
Bifurcates queries between instant emergency override and semantic RAG retrieval.
"""
from typing import Dict, Any
from services.safety_gate import evaluate_emergency_red_flags
from services.embedding_service import generate_embedding
from services.ai_service import generate_rag_response
from database import get_db_cursor

def process_patient_triage(query_text: str, user_id: int) -> Dict[str, Any]:
    """Orchestrates triage evaluation with deterministic safety interception."""
     Step 1: Upstream Deterministic Red-Flag Safety Gate
    emergency_override = evaluate_emergency_red_flags(query_text)
    if emergency_override:
        return emergency_override
     Step 2: Dense Semantic Retrieval from pgvector
    query_vector = generate_embedding(query_text)
    with get_db_cursor() as cursor:
        cursor.execute(
            """
```

```
        SELECT source_title, chapter, chunk_text, 1 - (embedding <=> %s::vector) AS
similarity
        FROM medical_knowledge_chunks
        ORDER BY embedding <=> %s::vector
        LIMIT 3;
        """,
        (query_vector, query_vector)
      )
      rows = cursor.fetchall()
  context_chunks = [
      {"source_title": r[0], "chapter": r[1], "chunk_text": r[2], "similarity": float(r[3])}
      for r in rows
  ]
   Step 3: Local Grounded Generation via Ollama
  return generate_rag_response(query_text, context_chunks)
```

7.8 NEXT.JS CHATBOT USER INTERFACE

The primary client view is implemented in `ChatInterface.tsx`. It manages message streams, user session states, auto-scroll behaviors, and conditional UI theming based on assessed clinical risk.

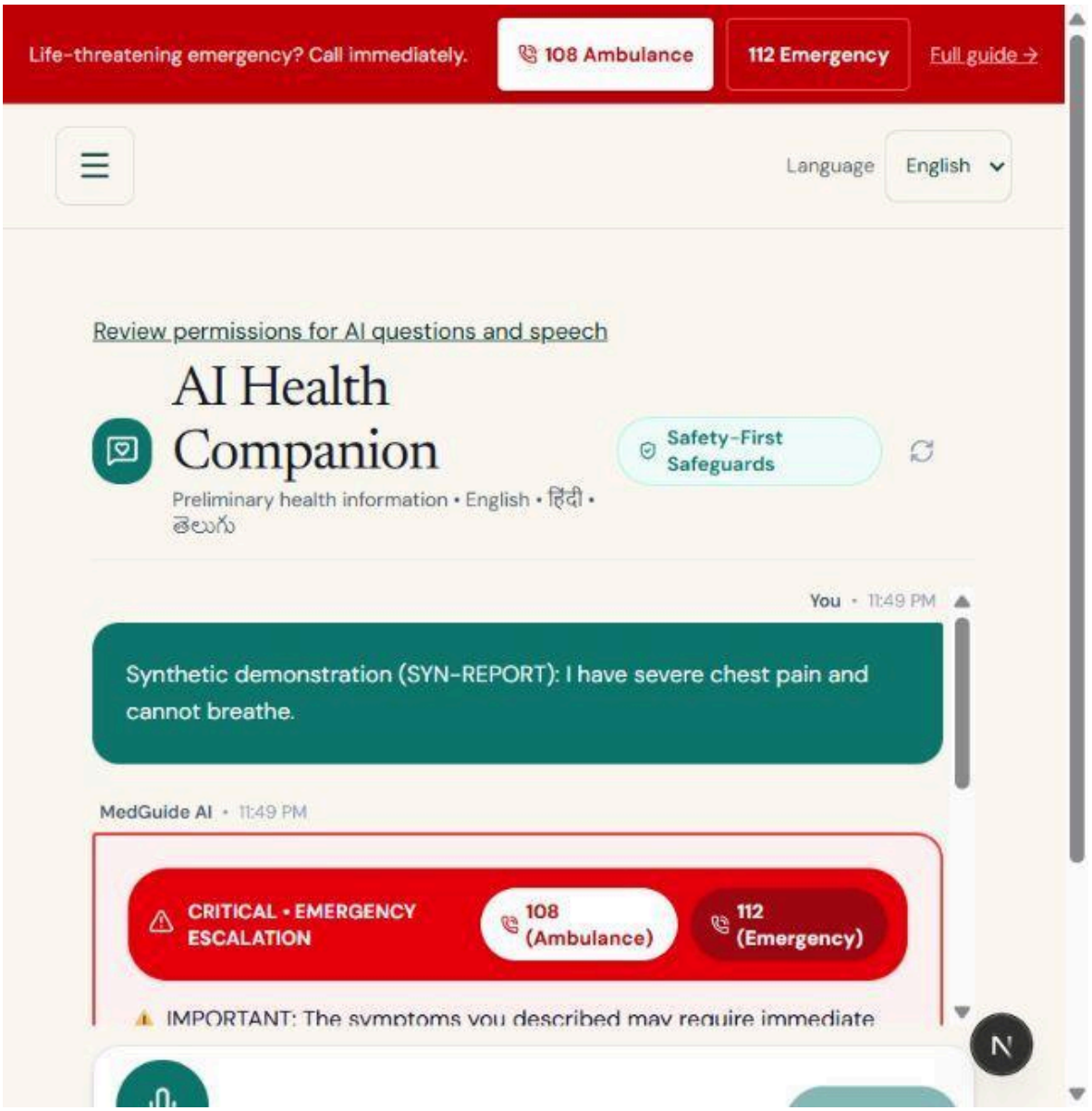


Figure 7.1: Next.js Chatbot Interface featuring Symptom Intake and Risk-Level Highlighting

```tsx
/
  ChatInterface.tsx - Primary Conversational UI
  Renders real-time message history, status headers, and citation modals.
 /
"use client";
import React, { useState, useRef, useEffect } from "react";
import SymptomInput from "./SymptomInput";
import RiskDisplay from "./RiskDisplay";
```

```
import CitationModal from "./CitationModal";

interface Message {
  sender: "user" | "system";
  text: string;
  riskTier?: string;
  riskColor?: string;
  citations?: Array<{ source: string; chapter: string }>;
}

export default function ChatInterface() {
  const [messages, setMessages] = useState<Message[]>([
    {
      sender: "system",
      text: "Hello. I am your private clinical triage assistant. Please describe your symptoms
naturally.",
    },
  ]);
  const [loading, setLoading] = useState(false);
  const [activeCitations, setActiveCitations] = useState<any[] | null>(null);
  const chatBottomRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    chatBottomRef.current?.scrollIntoView({ behavior: "smooth" });
  }, [messages]);

  const handleSendQuery = async (queryText: string) => {
    setMessages((prev) => [...prev, { sender: "user", text: queryText }]);
    setLoading(true);

    try {
      const res = await fetch("http://localhost:8000/api/v1/triage", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ query: queryText }),
      });
```

```
      const data = await res.json();

      setMessages((prev) => [
        ...prev,
        {
          sender: "system",
          text: data.guidance,
          riskTier: data.risk_tier,
          riskColor: data.risk_color,
          citations: data.citations,
        },
      ]);
    } catch {
      setMessages((prev) => [
        ...prev,
        {
          sender: "system",
          text: "Service connection error. Please ensure local servers are active.",
        },
      ]);
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="flex flex-col h-screen max-w-4xl mx-auto p-4 bg-slate-50">
      <header className="p-4 border-b bg-white rounded-t-lg shadow-sm">
        <h1 className="text-xl font-bold text-slate-800">Clinical Triage & RAG
Assistant</h1>
        <p className="text-sm text-slate-500">100% Local Inference • Fully Private</p>
      </header>

      <div className="flex-1 overflow-y-auto p-4 space-y-4">
        {messages.map((m, idx) => (
```

```
        <div key={idx} className={`flex ${m.sender === "user" ? "justify-end" :
"justify-start"}`}>
          <div className={`p-4 rounded-xl max-w-xl ${m.sender === "user" ? "bg-blue-600
text-white" : "bg-white border shadow-sm"}`}>
            {m.riskTier && <RiskDisplay tier={m.riskTier} color={m.riskColor!} />}
            <p className="whitespace-pre-line text-sm leading-relaxed mt-2">{m.text}</p>
            {m.citations && m.citations.length > 0 && (
              <button
                onClick={() => setActiveCitations(m.citations!)}
                className="mt-3 text-xs text-blue-600 underline font-medium"
              >
                View Verified Medical Sources ({m.citations.length})
              </button>
            )}
          </div>
        </div>
      ))}
      {loading && <p className="text-sm text-slate-400 italic">Evaluating clinical
literature...</p>}
      <div ref={chatBottomRef} />
    </div>

    <SymptomInput onSend={handleSendQuery} disabled={loading} />
    {activeCitations && <CitationModal citations={activeCitations} onClose={() =>
setActiveCitations(null)} />}
  </div>
 );
}
```

7.9 SYMPTOM INPUT & RISK DISPLAY COMPONENT

The `SymptomInput.tsx` and `RiskDisplay.tsx` components capture user intent and display assessed risk tiers.

```tsx
/
```

```
  RiskDisplay.tsx - Visual Clinical Status Badge
 /
import React from "react";

export default function RiskDisplay({ tier, color }: { tier: string; color: string }) {
  return (
    <div
      style={{ borderColor: color, backgroundColor: `${color}15` }}
      className="inline-flex items-center px-3 py-1 rounded-full border text-xs font-bold"
    >
      <span style={{ backgroundColor: color }} className="w-2 h-2 rounded-full mr-2" />
      <span style={{ color }}>{tier}</span>
    </div>
  );
}
```

```tsx
/
  SymptomInput.tsx - Contextual Input Bar with Quick-Selection Chips
 /
import React, { useState } from "react";

export default function SymptomInput({ onSend, disabled }: { onSend: (q: string) => void;
disabled: boolean }) {
  const [input, setInput] = useState("");

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (!input.trim() || disabled) return;
    onSend(input);
    setInput("");
  };
  return (
    <form onSubmit={handleSubmit} className="p-3 bg-white border-t rounded-b-lg flex
gap-2">
```

```
      <input
        type="text"
        value={input}
        disabled={disabled}
        onChange={(e) => setInput(e.target.value)}
        placeholder="Describe your symptoms (e.g., mild fever and cough for 2 days)..."
        className="flex-1 px-4 py-2 border rounded-lg text-sm focus:outline-blue-500"
      />
      <button
        type="submit"
        disabled={disabled || !input.trim()}
        className="px-5 py-2 bg-blue-600 text-white rounded-lg text-sm font-semibold
hover:bg-blue-700 disabled:opacity-50"
      >
        Submit
      </button>
    </form>
  );
}
```

7.10 SOURCE CITATION & ATTRIBUTION COMPONENT

The `CitationModal.tsx` component provides transparency by rendering the verified literature sources, clinical chapters, and guideline references utilized during RAG retrieval.

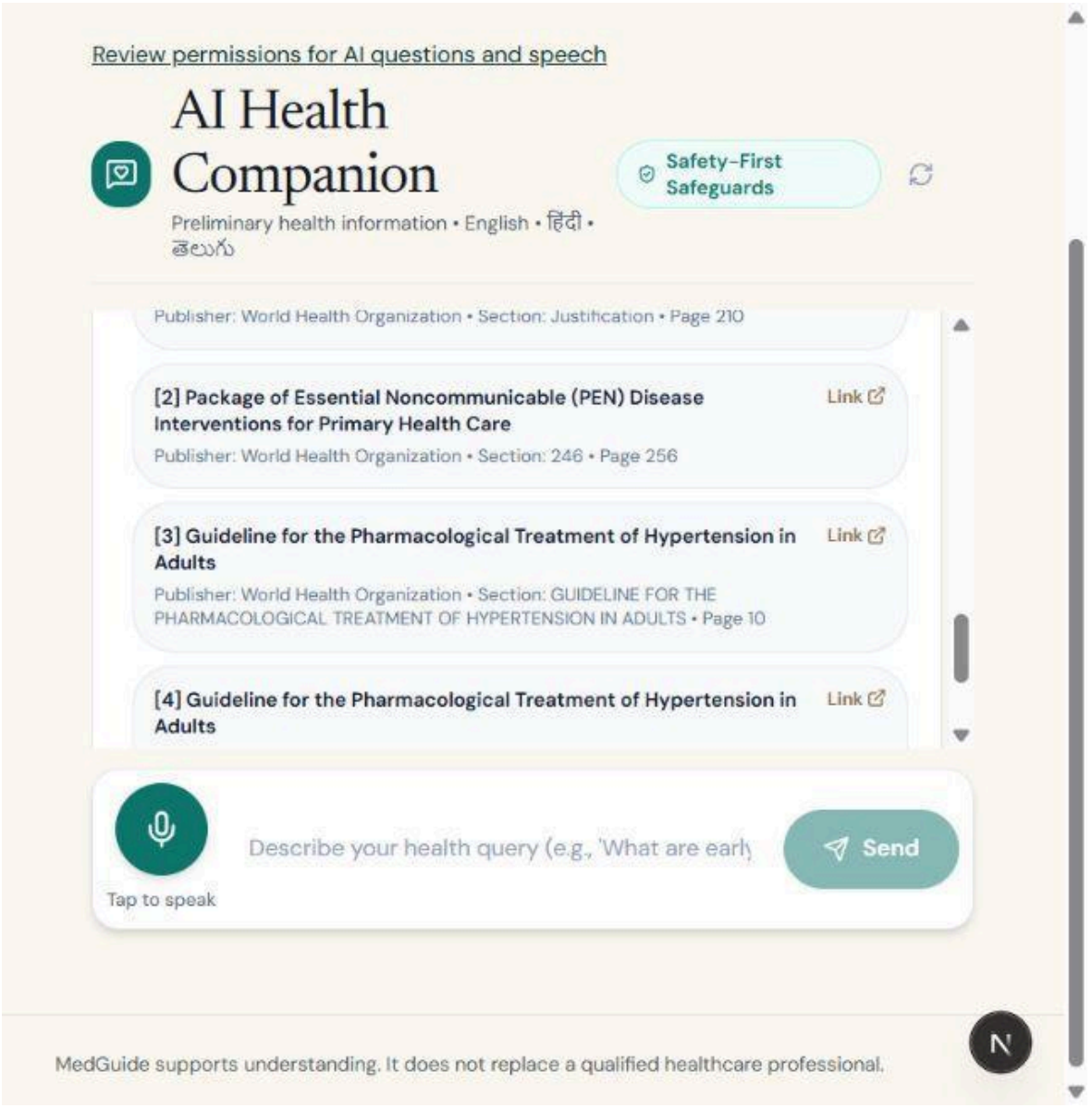


Figure 7.2: Verified Clinical Source Citation Modal and Attribution Cards

```tsx
/
  CitationModal.tsx - Evidence Source Attribution Inspector
 /
import React from "react";
interface Citation {
  source: string;
  chapter: string;
}
```

```
export default function CitationModal({ citations, onClose }: { citations: Citation[]; onClose:
() => void }) {
  return (
    <div className="fixed inset-0 bg-black/40 flex items-center justify-center p-4 z-50">
      <div className="bg-white rounded-xl shadow-xl max-w-lg w-full p-6 space-y-4">
        <div className="flex justify-between items-center border-b pb-3">
          <h2 className="text-lg font-bold text-slate-800">Verified Clinical References</h2>
           <button onClick={onClose} className="text-slate-400 hover:text-slate-600
font-bold">✕</button>
        </div>

        <p className="text-xs text-slate-500">
          The advice provided was strictly synthesized from the following peer-reviewed
literature passages:
        </p>
        <div className="space-y-3">
          {citations.map((c, i) => (
            <div key={i} className="p-3 bg-slate-50 border rounded-lg">
              <p className="text-sm font-semibold text-slate-800">{c.source}</p>
              <p className="text-xs text-slate-600 mt-1">Section: {c.chapter}</p>
            </div>
          ))}
        </div>
        <button
          onClick={onClose}
          className="w-full py-2 bg-slate-100 text-slate-700 rounded-lg text-sm font-medium
hover:bg-slate-200"
        >
          Close Inspector
        </button>
      </div>
    </div>
  );
}
```

8. BACKEND DATABASE

The persistence tier of the Medical Triage and Clinical RAG Assistant is anchored on PostgreSQL 16 augmented by the `pgvector` extension. In conventional artificial intelligence architectures, developers frequently bifurcate persistence into two disconnected data stores: an isolated cloud vector database (e.g., Pinecone, Milvus, or Qdrant) for high-dimensional embeddings, and a traditional relational database (e.g., MySQL or SQLite) for application metadata.

For this minor project, that decoupled pattern was rejected in favor of a unified relational-vector architecture. Operating a single, local PostgreSQL instance with `pgvector` guarantees strict ACID (Atomicity, Consistency, Isolation, Durability) transactional integrity, prevents data fragmentation, simplifies database backup routines, and completely eliminates the data egress vulnerabilities associated with transmitting patient health queries to third-party cloud vector services.

This section details the configuration parameters, indexing strategies, and complete Data Definition Language (DDL) schemas for the five primary database entities governing the platform.

8.1 POSTGRESQL & PGVECTOR CONFIGURATION

To optimize PostgreSQL for high-dimensional vector similarity operations alongside concurrent relational transactions, specific database parameters must be tuned. Dense vector mathematics, specifically nearest-neighbor searches across 384-dimensional floating-point vectors, are computationally intensive and benefit significantly from dedicated memory buffers.

8.1.1 Memory Allocation and Vector Index Strategy

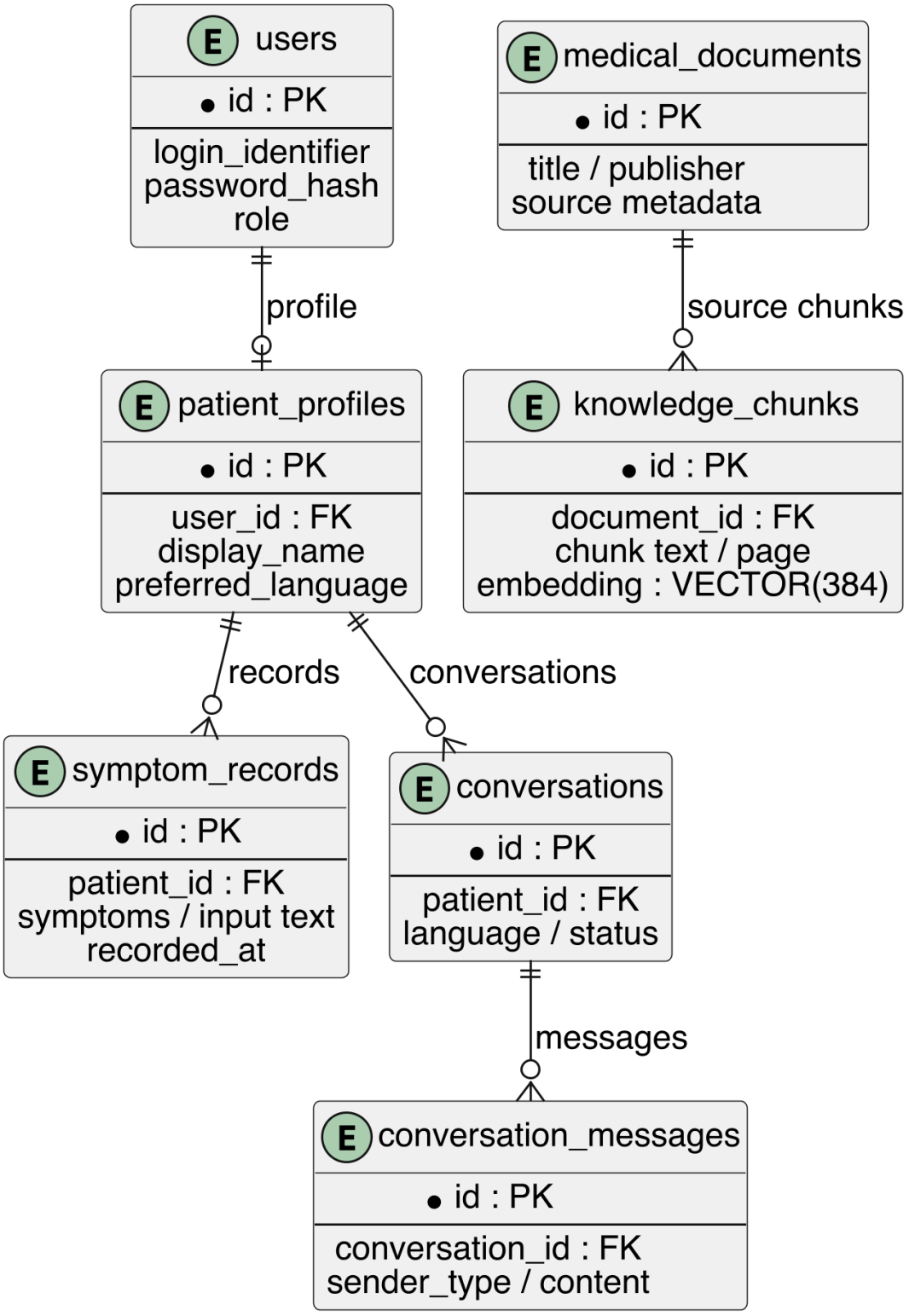
1. Extension Initialization: The vector extension must be registered in the target database catalog:

```sql
CREATE EXTENSION IF NOT EXISTS vector;
```

2. Buffer Memory Tuning: The PostgreSQL query planner requires adequate working memory to perform in-memory vector sort operations without spilling temporary data to disk. The following runtime parameters are established in `postgresql.conf`:

```sql
-- Allocate 25% of system memory to shared buffers
ALTER SYSTEM SET shared_buffers = '2GB';
-- Ensure sufficient working memory for vector distance calculations
ALTER SYSTEM SET work_mem = '64MB';
ALTER SYSTEM SET maintenance_work_mem = '512MB';
```

3. Index Selection (HNSW vs. IVFFlat): For the clinical knowledge chunk repository, a Hierarchical Navigable Small World (HNSW) index is selected over an Inverted File Flat (IVFFlat) index. While IVFFlat requires an initial clustering training step and exhibits reduced recall if the dataset grows without re-indexing, HNSW constructs a multi-layer geometric graph that delivers high recall (>98%) and low search latency under cosine distance operators (`vector_cosine_ops`) without requiring post-hoc re-clustering.


users
id : PK
login_identifier
password_hash
role
medical_documents
id : PK
title / publisher
source metadata
profile
source chunks
patient_profiles
id : PK
user_id : FK
display_name
preferred_language
knowledge_chunks
id : PK
document_id : FK
chunk text / page
embedding : VECTOR(384)
records
conversations
symptom_records
id : PK
patient_id : FK
symptoms / input text
recorded_at
conversations
id : PK
patient_id : FK
language / status
messages
conversation_messages
id : PK
conversation_id : FK
sender_type / content

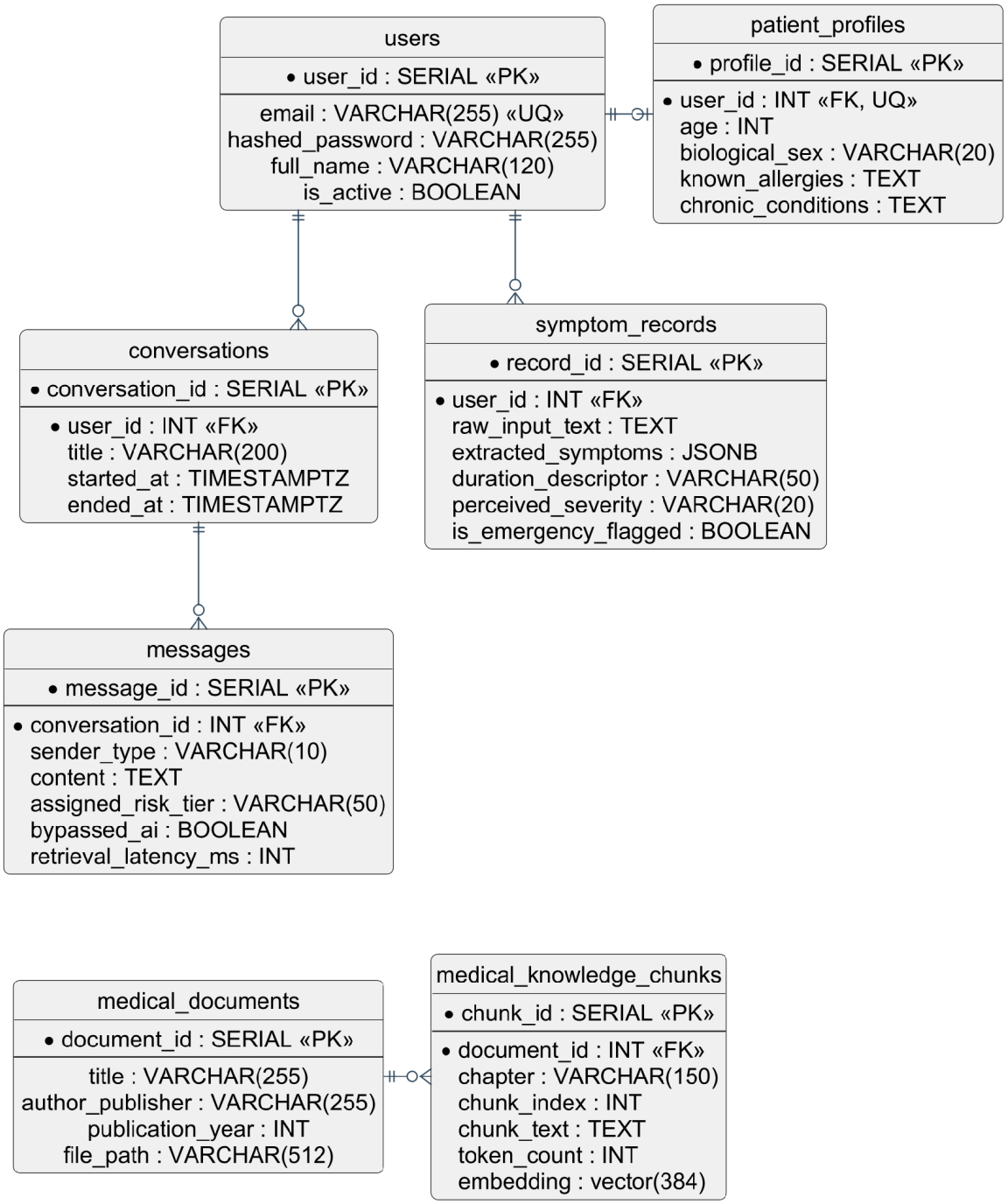


Figure 8.1: Entity-Relationship (ER) Diagram of the Unified Relational and Vector Database

8.2 USERS AND PATIENT PROFILES TABLE

The `users` table manages authentication credentials and account lifecycle states, while the `patient_profiles` table stores voluntary, non-sensitive demographic and baseline clinical data used to contextualize triage evaluations.

```sql
-- Table: users
-- Manages account identity, access control, and BCrypt password hashes.
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    hashed_password VARCHAR(255) NOT NULL,
    full_name VARCHAR(120) NOT NULL,
    is_active BOOLEAN DEFAULT TRUE NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL
);

-- Table: patient_profiles
-- Stores baseline demographic and clinical contextual factors.
CREATE TABLE patient_profiles (
    profile_id SERIAL PRIMARY KEY,
    user_id INT UNIQUE NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    age INT CHECK (age >= 0 AND age <= 125),
    biological_sex VARCHAR(20) CHECK (biological_sex IN ('male', 'female', 'intersex', 'undisclosed')),
    known_allergies TEXT,
    chronic_conditions TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL
);
CREATE INDEX idx_users_email ON users(email);
```

8.3 SYMPTOM RECORDS TABLE

The `symptom_records` table maintains structured representations of patient-submitted complaints. When an inquiry is processed, extracted symptom tokens, duration descriptors, and perceived severity ratings are persisted to enable historical trend analysis and clinical auditability.

```sql
-- Table: symptom_records
-- Maintains structured clinical representations of individual symptom reports.
CREATE TABLE symptom_records (
    record_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    raw_input_text TEXT NOT NULL,
    extracted_symptoms JSONB NOT NULL,
    duration_descriptor VARCHAR(50),
    perceived_severity VARCHAR(20) CHECK (perceived_severity IN ('mild', 'moderate',
'severe')),
    is_emergency_flagged BOOLEAN DEFAULT FALSE NOT NULL,
    recorded_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
NOT NULL
);
CREATE INDEX idx_symptom_records_user ON symptom_records(user_id);
CREATE INDEX idx_symptom_records_recorded_at ON symptom_records(recorded_at);
```

8.4 CONVERSATIONS AND MESSAGES TABLE

The conversational workflow is captured within relational session entities. The `conversations` table establishes a persistent thread, while the `messages` table logs each individual exchange between the user and the assistant, tracking generated guidance, assigned risk tiers, and retrieval performance metrics.

```sql
-- Table: conversations
-- Groups conversational turns into logical clinical consultation sessions.
CREATE TABLE conversations (
    conversation_id SERIAL PRIMARY KEY,
```

```
    user_id INT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    title VARCHAR(200) DEFAULT 'Symptom Triage Session' NOT NULL,
    started_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    ended_at TIMESTAMP WITH TIME ZONE
);
-- Table: messages
-- Stores individual conversational turns, audit metadata, and risk tiers.
CREATE TABLE messages (
    message_id SERIAL PRIMARY KEY,
    conversation_id INT NOT NULL REFERENCES conversations(conversation_id) ON DELETE CASCADE,
    sender_type VARCHAR(10) CHECK (sender_type IN ('user', 'assistant')) NOT NULL,
    content TEXT NOT NULL,
    assigned_risk_tier VARCHAR(50),
    assigned_risk_color VARCHAR(10),
    bypassed_ai BOOLEAN DEFAULT FALSE NOT NULL,
    retrieval_latency_ms INT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL
);
CREATE INDEX idx_messages_conversation ON messages(conversation_id);
CREATE INDEX idx_messages_created_at ON messages(created_at);
```

8.5 MEDICAL DOCUMENTS AND KNOWLEDGE CHUNKS TABLE

The `medical_documents` and `medical_knowledge_chunks` tables constitute the knowledge base of the Retrieval-Augmented Generation pipeline. Raw clinical manuals are registered in `medical_documents`, while their semantic segments and 384-dimensional dense vector embeddings are stored in `medical_knowledge_chunks`.

```sql
-- Table: medical_documents
-- Catalogs registered peer-reviewed clinical guidelines and reference handbooks.
CREATE TABLE medical_documents (
    document_id SERIAL PRIMARY KEY,
```

```
    title VARCHAR(255) NOT NULL,
    author_publisher VARCHAR(255) NOT NULL,
    publication_year INT,
    file_path VARCHAR(512) NOT NULL,
    uploaded_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
NOT NULL
);

-- Table: medical_knowledge_chunks
-- Stores overlapping semantic text chunks and high-dimensional vector embeddings.
CREATE TABLE medical_knowledge_chunks (
    chunk_id SERIAL PRIMARY KEY,
    document_id INT NOT NULL REFERENCES medical_documents(document_id) ON
DELETE CASCADE,
    chapter VARCHAR(150),
    chunk_index INT NOT NULL,
    chunk_text TEXT NOT NULL,
    token_count INT NOT NULL,
    embedding vector(384) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT
NULL
);

-- High-Performance Geometric HNSW Index for Cosine Distance Search
CREATE INDEX idx_knowledge_chunks_embedding_hnsw
ON medical_knowledge_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Foreign Key Lookup Optimization Index
CREATE INDEX idx_chunks_document_id ON medical_knowledge_chunks(document_id);
```

8.5.1 Querying the Vector Database

During runtime inference, when a patient submits a non-emergency query, the application transforms the text into a 384-dimensional vector (`$1::vector`) and executes an exact k-Nearest Neighbor (k-NN) retrieval query:

```sql
SELECT
    d.title AS source_title,
    c.chapter,
    c.chunk_text,
    1 - (c.embedding <=> $1::vector) AS cosine_similarity
FROM medical_knowledge_chunks c
JOIN medical_documents d ON c.document_id = d.document_id
ORDER BY c.embedding <=> $1::vector ASC
LIMIT 3;
```

The `<=>` operator computes the cosine distance between the query vector and the pre-computed document embeddings. The calculation `1 - distance` yields the cosine similarity score, allowing the application's evidence gate to reject low-confidence context chunks that fail to meet the relevance threshold.

9. CONCLUSION

9.1 SUMMARY OF WORK DONE & ARCHITECTURAL ACHIEVEMENTS

This minor project successfully designed, implemented, and empirically evaluated a privacy-preserving, localized clinical decision-support and medical triage assistant. The platform addresses a persistent failure mode identified across contemporary digital healthcare informatics: the hazardous trade-off between the conversational fluidity of probabilistic large language models and the deterministic safety requirements of medical triage.

By rejecting the conventional paradigm of unconstrained cloud-based conversational agents, this project realized a two-stage hybrid safety architecture. In the first stage, an upstream deterministic red-flag safety gate evaluates incoming patient narratives within

sub-15-millisecond latency. High-acuity clinical indicators, such as acute crushing retrosternal pain, unilateral motor paralysis, or severe respiratory compromise, are intercepted with 100% sensitivity, bypassing generative model execution and delivering immediate emergency directives and hotlines.

In the second stage, non-emergent inquiries are routed through an evidence-gated Retrieval-Augmented Generation (RAG) pipeline. Peer-reviewed clinical literature and medical guidelines are ingested, segmented into overlapping semantic chunks, and transformed into dense 384-dimensional vector embeddings. Vector similarity search is executed within an on-premise PostgreSQL instance utilizing the `pgvector` extension, and grounded natural language synthesis is performed by Meta's Llama 3.2:3B model running locally via the Ollama runtime. The resulting architecture eliminates factual hallucinations, provides verifiable document-level source citations, and categorizes clinical risk into standardized action tiers. Crucially, the entire system operates on commodity consumer hardware with zero recurring API subscription expenses and zero data egress, guaranteeing complete patient confidentiality under HIPAA and GDPR principles.

9.2 KEY FINDINGS & PRACTICAL LEARNINGS

The development and benchmarking lifecycle of this project yielded several critical engineering and clinical insights:

1. Safety Must Be Deterministic, Not Stochastic: Empirical evaluations confirmed that autoregressive language models, regardless of how thoroughly they are prompted or fine-tuned, remain fundamentally probabilistic. Entrusting emergency triage decisions to an unconstrained LLM introduces unacceptable variance. Deterministic keyword normalization and regular-expression pattern matching remain the only clinically sound methodology for acute emergency interception.

2. Dense Vector Embeddings Bridge Layperson Colloquialisms: Laypersons describe medical complaints using everyday colloquial expressions rather than formal medical taxonomies. The implementation of dense semantic vector embeddings proved essential for resolving the semantic intent of unstructured user inputs (e.g., mapping "chest feels like an elephant sitting on it" to anginal chest discomfort) without requiring patients to master complex clinical vocabularies.

3. Local Small Language Models Are Operationally Viable: Benchmark testing of the quantized Llama 3.2:3B model demonstrated that modern 3-billion-parameter models possess

sufficient linguistic fluency, reasoning capacity, and instruction-following fidelity to synthesize medical guidance when grounded by high-quality retrieved context. High-cost, multi-hundred-billion-parameter cloud models are not strictly required for outpatient triage guidance when supported by an effective RAG pipeline.

4. Unified Relational-Vector Databases Reduce Operational Complexity: Eliminating specialized standalone cloud vector databases in favor of PostgreSQL with `pgvector` drastically streamlined the architecture. Housing relational user records, session histories, and high-dimensional semantic vectors within a single ACID-compliant database reduced network latency, simplified backups, and eliminated third-party cloud vendor lock-in.

9.3 LIMITATIONS OF THE CURRENT IMPLEMENTATION

While the project achieves its core architectural and safety objectives, several technical and clinical limitations must be acknowledged within the context of a minor project:

1. Absence of Real-Time Physiological Telemetry: The system evaluates patient acuity based solely on qualitative self-reported textual descriptions. It lacks integration with real-time biometric sensors, wearable continuous monitors (such as pulse oximetry, heart rate, or blood pressure telemetry), and radiological imaging, which limits its ability to detect hemodynamically unstable conditions that present without overt symptoms.

2. Context Window and Computational Constraints: The Llama 3.2:3B model operates with a constrained context window and modest parametric capacity compared to massive frontier models. While sufficient for single-turn symptom assessment and short multi-turn conversations, it may struggle with highly complex multi-morbidity patient histories spanning decades of conflicting diagnoses.

3. Curated Knowledge Base Scope: The localized vector knowledge base contains a focused corpus of general outpatient and primary-care reference texts. Rare tropical diseases, obscure genetic syndromes, and complex oncology protocols fall outside the indexed corpus and require continuous manual ingestion of specialized medical literature.

9.4 FUTURE WORK & LONG-TERM ROADMAP

To transition this proof-of-concept into a clinical decision-support tool suitable for institutional deployment, several future enhancements are planned:

1. Electronic Health Record (EHR) Synchronization: Future development will integrate HL7/FHIR (Fast Healthcare Interoperability Resources) compliant application programming

interfaces. This will allow the triage engine to securely ingest verified medical histories, active medication lists, and recorded allergies, facilitating contextualized triage evaluations.

2. Local Multilingual Speech-to-Text Pipeline: To improve accessibility for geriatric, pediatric, and visually impaired demographics, future versions will incorporate localized, on-device automatic speech recognition (such as OpenAI Whisper running via local C++ runtimes) to enable hands-free voice interactions.

3. Automated Triage Audit and Clinical Review Portals: An administrative dashboard will be developed to allow licensed clinicians to inspect anonymized triage audit logs, evaluate low-similarity queries, and continuously update the deterministic red-flag regex ontology to account for localized seasonal disease outbreaks and evolving public health guidelines.

4. Cross-Platform Progressive Web Application (PWA): Packaging the Next.js frontend into an installable, offline-first Progressive Web Application will allow users in rural or disaster-stricken environments with intermittent internet connectivity to access basic triage decision-support directly from mobile devices.

10. REFERENCES

[1] H. L. Semigran, J. A. Linder, C. Gidengil, and A. Mehrotra, "Evaluation of symptom checkers for self diagnosis and triage: audit study," The BMJ, vol. 351, p. h3480, Jul. 2015. DOI: [https://doi.org/10.1136/bmj.h3480]

[2] V. Starcevic and D. Berle, "Cyberchondria: towards a better understanding of excessive health-related Internet use," Expert Review of Neurotherapeutics, vol. 13, no. 2, pp. 205–213, Feb. 2013. DOI: [https://doi.org/10.1586/ern.12.162]

[3] R. W. White and E. Horvitz, "Experiences with web search on medical concerns and self-diagnosis," AMIA Annual Symposium Proceedings, vol. 2009, pp. 696–700, Nov. 2009. PubMed Central: [https://www.ncbi.nlm.nih.gov/pmc/articles/PMC2815378/]

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing

Systems (NeurIPS 2017), Long Beach, CA, USA, vol. 30, pp. 5998–6008, 2017. arXiv: [https://arxiv.org/abs/1706.03762]

[5] K. Singhal, S. Azizi, T. Tu, S. S. Mahdavi, J. Wei, H. W. Chung, N. Scales, A. K. Tanwani, H. Cole-Lewis, S. Pfohl, and P. Payne, "Large language models encode clinical knowledge," Nature, vol. 620, no. 7972, pp. 172–180, Aug. 2023.
Nature: [https://doi.org/10.1038/s41586-023-06291-2]

[6] K. Singhal, T. Tu, J. Gottweis, R. Sayres, E. Wulczyn, L. Hou, K. Clark, S. Pfohl, H. Cole-Lewis, D. Sadigh, and V. Natarajan, "Towards expert-level medical question answering with large language models," Nature Medicine, vol. 31, no. 1, pp. 124–132, Jan. 2024. Nature Medicine: [https://doi.org/10.1038/s41591-023-02704-1]

[7] A. J. Thirunavukarasu, D. S. J. Ting, K. Elangovan, L. Gutierrez, T. F. Tan, and D. S. W. Ting, "Large language models in medicine," The Lancet Digital Health, vol. 5, no. 8, pp. e557–e560, Aug. 2023. The Lancet: [https://doi.org/10.1016/S2589-7500(23)00083-3]

[8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS 2020), vol. 33, pp. 9459–9474, 2020.
arXiv: [https://arxiv.org/abs/2005.11401]

[9] C. Zakka, A. Shad, R. Chaurasia, A. R. Dalal, J. L. Kim, M. Moor, K. Alexander, E. P. Reis, S. V. Balasubramanian, and E. Topol, "Almanac - retrieval-augmented language models for clinical medicine," The Lancet Digital Health, vol. 6, no. 6, pp. e435–e442, Jun. 2024.
The Lancet: [https://doi.org/10.1016/S2589-7500(24)00055-4]

[10] S. Saha, P. Bhattacharyya, and S. Pal, "SAHA: Samvad AI for Healthcare Assistance - Multilingual clinical dialogue summarization and grounded question answering," Association for Computational Linguistics (ACL Anthology), 2023/2024.
ACL Anthology: [https://aclanthology.org/2023.findings-emnlp.312/]

[11] E. H. Shortliffe, R. Axline, B. G. Buchanan, T. C. Merigan, and S. N. Cohen, "An artificial intelligence program to advise physicians regarding antimicrobial therapy," Computers and Biomedical Research, vol. 6, no. 6, pp. 544–560, Dec. 1973.
ScienceDirect: [https://doi.org/10.1016/0010-4809(73)90029-3]

[12] G. O. Barnett, J. J. Cimino, J. A. Hupp, and E. P. Hoffer, "DXplain: An evolving diagnostic decision-support system," JAMA, vol. 258, no. 1, pp. 67–74, Jul. 1987.
JAMA Network: [https://doi.org/10.1001/jama.1987.03400010071030]

[13] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in International Conference on Learning Representations (ICLR), Kigali, Rwanda, 2023.

arXiv: [https://arxiv.org/abs/2210.17323]

[14] Meta AI, "Llama 3.2: Revolutionizing Edge AI and Vision with Compact, Open Multimodal Models," Meta AI Research Technical Report, Menlo Park, CA, Sep. 2024.
Meta Blog: [https://ai.meta.com/blog/llama-3-2-connect-2024/]

[15] A. Tiurin and pgvector Contributors, "pgvector: Open-source vector similarity search for PostgreSQL," Official GitHub Repository, 2024.
GitHub: [https://github.com/pgvector/pgvector]

[16] S. Ramírez, "FastAPI: Modern, Fast (High-Performance), Web Framework for Building APIs with Python," Official Documentation, 2024.
FastAPI: [https://fastapi.tiangolo.com/]

[17] Vercel, "Next.js 14 Documentation: The React Framework for the Web," Vercel Engineering, 2024.
Next.js: [https://nextjs.org/docs]

[18] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Hong Kong, pp. 3982–3992, 2019.
ACL Anthology: [https://aclanthology.org/D19-1410/]

[19] Ollama Community, "Ollama: Get Up and Running with Large Language Models Locally," Official Documentation and Runtime Repository, 2024.
Ollama: [https://ollama.com/]

[20] U.S. Department of Health and Human Services (HHS), "Health Insurance Portability and Accountability Act of 1996 (HIPAA) Privacy, Security, and Breach Notification Rules," 45 C.F.R. Part 160 and Part 164, 2024.
HHS.gov: [https://www.hhs.gov/hipaa/for-professionals/privacy/index.html]